



西南科技大学

Southwest University of Science and Technology

学术型硕士研究生学位论文

ACADEMIC MASTER THESIS

(工学硕士 Master of Engineering)

Climate Forecasting System Based on Deep Learning and Multi-Sensors

作者姓名 Uddin Sahab

学号 7420240311

导师 何刚

学科 Software Engineering

答辩委员会成员

评阅人

2025 年 12 月

分类号:

学校代码: 10619

UDC:

密级:

西南科技大学硕士学位论文

(理学硕士 Master of Science)

Climate Forecasting System Based on Deep Learning and Multi-Sensors

作者姓名: UDDIN SAHAB

学科(专业)或领域名称: 软件工程(人工智能)

指导教师姓名、职称: 何刚(副教授)

校外指导教师姓名、职称:

学生所属学院名称: 学校计算机科学与技术

论文答辩日期: 2025 年 12 月

A Thesis of Southwest University of Science and Technology for the
Degree of Master
(Master of Natural Science)

Climate Forecasting System Based on Deep Learning and Multi-Sensors

Candidate: UDDIN SAHAB

Supervisor: 何刚

12/ 2025

学位论文独创性声明

本人郑重声明：所呈交的学位论文是我个人在导师的指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明，并表示了谢意。本人依法享有和承担由此论文所产生的权利和责任。

Statement in originality of thesis

I solemnly declare that the submitted thesis is the research work and achievements I have conducted under the guidance of my supervisor. To the best of my knowledge, the paper does not include any research results that have been published or written by others, except for those specifically marked and acknowledged in the text. My colleagues who work with me have provided clear explanations and expressed gratitude for any contributions made to this research institute in the paper. I enjoy and assume the rights and responsibilities arising from this paper in accordance with the law.

论文作者签名：

Graduate Signature:

日期： 年 月 日

Date:

学位论文版权使用授权声明

本学位论文作者完全了解学校有关保存、使用学位论文的规定，即：研究生在西南科技大学学习和工作期间论文工作的知识产权单位属于西南科技大学。同意授权西南科技大学保留并向国家有关部门或机构送交该论文的印刷本和电子版本，允许该论文被查阅和借阅。同意授权西南科技大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印、扫描或数字化等其他复制手段保存和汇编本学位论文。保密论文在解密后遵守此规定。

Authorization Statement for Copyright Use of Thesis

The author of this thesis fully understands the regulations of the university regarding the preservation and use of the thesis, that is, the intellectual property unit of the graduate student's thesis work during their study and work at Southwest University of Science and Technology belongs to Southwest University of Science and Technology. Agree to authorize Southwest University of Science and Technology to retain and submit the printed and electronic versions of the paper to relevant national departments or institutions and allow the paper to be viewed and borrowed. Agree to authorize Southwest University of Science and Technology to include all or part of the content of this thesis in relevant databases for retrieval, and to use other copying methods such as photocopying, reduction, scanning, or digitization to save and compile this thesis. Confidential papers shall comply with this regulation after decryption.

论文作者签名：

Graduate Signature:

日 期： 年 月 日

Date:

指导导师签名：

Supervisor Signature:

日 期： 年 月 日

Date:

摘要

气候变化已成为 21 世纪最严峻的全球挑战之一，影响着生态系统、农业和人类生活。因此，准确的气候预测对于灾害管理、环境规划和可持续发展至关重要。传统的预测方法往往难以处理庞大而复杂的数据集，限制了其捕捉气候变量长期变化模式的能力。

本研究提出了一种基于深度学习的气候预测系统，旨在提高预测的准确性和适应性。该系统整合了云量、湿度、温度、降水、风速、辐射和气压等多种环境因素。我们开发并比较了多种神经网络模型——RNN、GRU、BiLSTM、CNN-LSTM 和 LSTM——用于预测次日最高气温。

所提出的 LSTM 模型通过 dropout 正则化增强以减少过拟合，在所有模型中取得了最高的准确率。实验结果表明，该模型能够有效捕捉时间依赖性，并提供稳定的预测结果，其平均绝对误差（MAE）和均方根误差（RMSE）均低于传统方法。

该模型部署在一个基于 Flask 的 Web 应用程序中，该应用程序采用 HTML、CSS、Bootstrap 和 JavaScript 构建，界面简洁易用。用户可以输入实时数据或上传 CSV 文件，即时获取预测结果和可视化图表。

本研究表明，将多传感器数据与先进的 LSTM 架构和适当的数据预处理相结合，可以显著提高现代气候预测系统的准确性和易用性。所提出的方法弥合了研究与实际应用之间的差距，为智能气候分析和决策支持提供了一种切实可行的解决方案。

关键词：气候预测系统、深度学习、多传感器数据、温度预测、LSTM、Flask 部署。

ABSTRACT

Climate change has become one of the most critical global challenges of the 21st century, affecting ecosystems, agriculture, and human life. Accurate climate forecasting is therefore essential for disaster management, environmental planning, and sustainable development. Traditional forecasting methods often struggle with large and complex datasets, limiting their ability to capture long-term patterns in climate variables.

This research presents a deep learning-based climate forecasting system designed to improve accuracy and adaptability. The system integrates multiple environmental factors such as humidity, temperature, precipitation, wind speed, radiation, and atmospheric pressure. Several neural network models RNN, GRU, BiLSTM, CNN-LSTM, and LSTM were developed and compared to predict next-day maximum temperature.

The proposed LSTM model, enhanced with dropout regularization to reduce overfitting, achieved the highest accuracy among all models. Experimental results show that it effectively captures temporal dependencies and provides stable predictions, with lower MAE and RMSE values than traditional approaches.

The model was deployed using a Flask-based web application, featuring an easy-to-use interface built with HTML, CSS, Bootstrap, and JavaScript. Users can input real-time data or upload CSV files to receive instant predictions and visualizations.

This study demonstrates that combining multi-sensor data with advanced LSTM architectures and proper data preprocessing can significantly enhance both the accuracy and usability of modern climate forecasting systems. The proposed approach bridges the gap between research and real-world application, offering a practical solution for intelligent climate analysis and decision support.

Keyword: Climate Forecasting System, Deep Learning, Multi-Sensor Data, Temperature Forecasting, LSTM, Flask Deployment.

Table of Contents

摘要	VI
ABSTRACT	VII
LIST OF FIGURES	XI
LIST OF TABLE	XIII
LIST OF ABBREVIATIONS	XIV
CHAPTER 1 INTRODUCTION	16
1.1 BACKGROUND & MOTIVATION	16
1.2 PROBLEM STATEMENT	18
1.3 RESEARCH OBJECTIVES	18
1.4 RESEARCH QUESTIONS	19
1.5 SCOPE OF THE STUDY	20
1.6 THESIS CONTRIBUTIONS	20
1.7 THESIS STRUCTURE	21
CHAPTER 2 LITERATURE REVIEW	23
2.1 CLIMATE FORECASTING APPROACHES	23
2.1.1 Traditional Approaches	23
2.1.2 Machine Learning Approaches	23
2.1.3 Deep Learning Approaches	24
2.1.4 Integrated and Hybrid Systems	24
2.2 DEEP LEARNING FOR TIME-SERIES FORECASTING	24
2.2.1 Recurrent Neural Networks (RNNs)	25
2.2.2 Long Short-Term Memory (LSTM)	25
2.2.3 Gated Recurrent Unit (GRU)	25
2.2.4 Bidirectional LSTM (BiLSTM)	25
2.2.5 Convolutional LSTM (CNN-LSTM)	25
2.2.6 Advantages of Deep Learning for Time-Series Data	26
2.3 MULTI-SENSOR WEATHER DATA IN PREDICTION	26
2.4 TRADITIONAL FORECASTING VS. DEEP LEARNING MODELS	27
2.5 DEPLOYMENT FRAMEWORKS FOR ML SYSTEMS	28
2.6 COMPARATIVE ANALYSIS OF EXISTING STUDIES	29
2.7 RESEARCH GAP & SUMMARY	31
CHAPTER 3 SYSTEM DESIGN	32
3.1 OVERALL SYSTEM CONCEPT	32
3.2 DATASET DESCRIPTION AND DATA SOURCES	33
3.3 KEY CLIMATE VARIABLES USED	36
3.4 DATA PREPARATION AND PREPROCESSING	37
3.5 SEQUENCE CONSTRUCTION AND LEAKAGE CONTROL	38
3.6 EVALUATION METRICS AND BASELINES	39
3.7 DEEP LEARNING MODELS FOR CLIMATE PREDICTION	40
3.7.1 Recurrent Neural Networks (RNNs)	40
3.7.2 Long Short-Term Memory (LSTM)	41
3.7.3 Gated Recurrent Unit (GRU)	43
3.7.4 Bidirectional Long Short-Term Memory (BiLSTM)	45
3.7.5 CNN-LSTM Hybrid Model	46

3.8 MODEL TRAINING SETUP	47
3.9 IMPLEMENTATION DETAILS	48
CHAPTER 4 RESEARCH METHODOLOGY	49
4.1 OVERVIEW OF METHODOLOGY	49
4.2 SYSTEM ARCHITECTURE & FLOWCHART	51
4.3 DATA PREPROCESSING	54
4.3.1 Data Cleaning and Outlier Handling	54
4.3.2 Missing-Value Treatment	54
4.3.3 Time Alignment and Resampling	54
4.3.4 Feature scaling	55
4.4 SEQUENCE CONSTRUCTION AND DATA SPLITTING	56
4.4.1 Sliding Window Generation	56
4.4.2 Chronological Train, Validation, and Test Split	56
4.4.3 Leakage Control and Sanity Checks	56
4.5 MODEL ARCHITECTURES	56
4.5.1 RNN Configuration	56
4.5.2 LSTM configuration	58
4.5.3 GRU Configuration	60
4.5.4 BiLSTM Configuration	61
4.5.5 CNN-LSTM Configuration	62
4.6 TRAINING PROTOCOL	64
4.6.1 Optimizer and learning-rate schedule	64
4.6.2 Early stopping and epoch limits	64
4.6.3 Reproducibility	64
4.7 HYPERPARAMETER TUNING	65
4.7.1 Search Space	65
4.7.2 Tuning Method and Selection Rule	66
4.7.3 Robustness Checks	66
4.8 IMPLEMENTATION DETAILS	67
4.8.1 Libraries and Environment	67
4.8.2 Hardware and Runtime	68
4.8.3 Code Structure and Version Control	68
4.9 DEPLOYMENT METHODOLOGY	70
4.9.1 Flask Backend Architecture	70
4.9.2 Login Interface and Authentication Table	71
4.9.3 API Endpoints and Input Validation	71
4.9.4 Model/Scaler Versioning and Logging	72
4.10 WEB INTERFACE WORKFLOW	72
4.10.1 Frontend Stack	73
4.10.2 Manual Input and CSV Upload Flow	74
4.10.3 Visualizations	75
CHAPTER 5 RESULTS AND DISCUSSION	77
5.1 EXPERIMENTAL SETUP	77
5.2 MODEL TRAINING AND PERFORMANCE VISUALIZATION	78

5.3 MODEL PERFORMANCE COMPARISON	80
5.4 ERROR ANALYSIS & DISCUSSION	81
5.5 MODEL EVALUATION AND COMPARISON	82
5.6 KEY OBSERVATIONS	84
5.7 VISUALIZATION OF RESULTS	84
CHAPTER 6 CONCLUSION AND FUTURE WORK	85
6.1 SUMMARY OF FINDINGS	85
6.2 PRACTICAL IMPLICATIONS	86
6.3 LIMITATIONS	87
6.4 FUTURE IMPROVEMENTS	87
ACKNOWLEDGEMENT	88
REFERENCES	89

List of Figures

Figure 1- 1 Climate Data Flow Overview	17
Figure 1- 2 Research Objectives Overview	19
Figure 1- 3 Research Motivation and Framework of the Climate Forecasting System .	21
Figure 2- 1 General Workflow of Traditional Climate Forecasting Process	27
Figure 3- 1 Overall System Flow	33
Figure 3- 2 The feature distributions exhibit natural variability	34
Figure 3- 3 The correlation heatmap of climate variable	35
Figure 3- 4 The time-series of temperature and humidity	35
Figure 3- 5 The STL decomposition of mean temperature	36
Figure 3- 6 LSTM Cell Architecture Diagram	42
Figure 3- 7 Neural Network Architecture	43
Figure 3- 8 CNN-LSTM Model Architecture	47
Figure 4- 1 Flowchart of the Climate Forecasting System Using Multi-Sensor Data and Cloud-Based Processing	51
Figure 4- 2 System Architecture Diagram	52
Figure 4- 3 System Architecture Flowchart	53
Figure 4- 4 RNN cells architectures diagrams	58
Figure 4- 5 Internal architecture of an LSTM	58
Figure 4- 6 Proposed LSTM Model architecture	59
Figure 4- 7 Internal architecture of a Gated Recurrent Unit	61
Figure 4- 8 The internal architecture of a BiLSTM	62
Figure 4- 9 CNN–LSTM Model Architecture	64
Figure 4- 10 Model Training and Evaluation Workflow	65
Figure 4- 11 Code Structure	69
Figure 4- 12 Login Interface and Authentication Table	71
Figure 4- 13 Workflow of User Interaction with the Web Interface	73
Figure 4- 14 User Login Interface	74

Figure 4- 15 User Input Interface	75
Figure 4- 16 Result Output and Location Map Interface	75
Figure 4- 17 Result Output and Location Map Interface	76
Figure 5- 1 Flask Web Interface Login Page (Development Mode)	78
Figure 5- 2 Training and Validation Loss (MSE) and MAE Curves of LSTM Model ...	79
Figure 5- 3 Combined Training and Validation Loss and MAE Curves Across Epochs	79
Figure 5- 4 Comparison of Evaluation Metrics for Different Models	81
Figure 5- 5 RMSE Comparison among RNN, LSTM, GRU, BiLSTM, and CNN-LSTM	83
Figure 5- 6 Predicted vs. Actual Temperature (Test Set)	83
Figure 5- 7 LSTM Forecast – True vs. Predicted Temperature Over Time	85

List of Table

Table 2- 1 The Comparative Analysis of Previous Studies	30
Table 3- 1 Dataset Overview	38
Table 4- 1 Key Improvements Introduced in the Proposed Model	60
Table 5- 1 The quantitative comparison of all models is summarized	80

List of Abbreviations

AI	Artificial Intelligence
ANN	Artificial Neural Network
API	Application Programming Interface
ARIMA	Auto-Regressive Integrated Moving Average
BiLSTM	Bidirectional Long Short-Term Memory
BN	Batch Normalization
CNN– LSTM	Convolutional Neural Network combined with Long Short-Term Memory
CSV	Comma-Separated Values
DL	Deep Learning
DNN	Deep Neural Network
EMA	Exponential Moving Average
GUI	Graphical User Interface
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
I/O	Input / Output
IoT	Internet of Things
JSON	JavaScript Object Notation
KNN	K-Nearest Neighbors
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error

N/A	Not Applicable
NLP	Natural Language Processing
NOAA	National Oceanic and Atmospheric Administration
PCA	Principal Component Analysis
R ²	Coefficient of Determination
RAM	Random Access Memory
ReLU	Rectified Linear Unit
RMSE	Root Mean Squared Error
RLR	ReduceLROnPlateau (Learning Rate Scheduler)
GAN	Generative Adversarial Network
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
GRU	Gated Recurrent Unit
SQL	Structured Query Language
SVM	Support Vector Machine
TF	TensorFlow
UI	User Interface
URL	Uniform Resource Locator
VS	Visual Studio
W/m ²	Watts per Square Meter
°C	Degree Celsius

Chapter 1 Introduction

Climate prediction is crucial for protecting lives and livelihoods, helping communities prepare for floods and heatwaves, assisting farmers with irrigation and harvest planning, and supporting sustainable resource management. However, the climate system is highly variable and nonlinear, and traditional forecasting methods based on fixed equations often fail to capture long-range patterns and complex interactions. As modern sensors generate more data, these limitations become even clearer in real-world applications.

This thesis develops a deep learning-based climate forecasting system that directly learns from multi-sensor data ^[1]. The system integrates temperature, humidity, wind speed, precipitation, pressure, and radiation into a consistent time-series pipeline. Five neural network models RNN, LSTM, GRU, BiLSTM, and CNN-LSTM are implemented and compared under identical conditions ^[2]. Using sliding-window sequences, the models capture both short-term fluctuations and long-term dependencies, with LSTM providing the most accurate and stable forecasts.

The dataset is carefully prepared to ensure fair and reproducible evaluation, with data cleaning, handling of gaps, and normalization ^[3]. Models are trained with a common optimizer and early stopping strategy to reduce overfitting, and performance is evaluated using MAE, RMSE, and R^2 .

Beyond modeling, the thesis delivers a fully realized engineering solution. The best-performing LSTM model is deployed using Flask, with a web interface built using HTML, CSS, Bootstrap, and JavaScript. Users can input values or upload CSV files for batch prediction, and the system displays clear charts and numerical outputs ^[4].

The goal is to provide a practical tool for short-term temperature forecasting that is transparent, reliable, and easy to extend ^[5]. By combining a rigorously evaluated LSTM model with an accessible web application, the system aids decision-making in agriculture, disaster preparedness, and climate services, demonstrating how deep learning and data handling can turn raw sensor data into actionable insights.

1.1 Background & Motivation

Extreme weather events are becoming more frequent and severe, posing significant challenges to human life, agriculture, and infrastructure ^[6]. Floods, heatwaves, and storms can devastate crops, disrupt transportation, and lead to economic losses ^[7]. Reliable short-term temperature forecasts are crucial in mitigating these impacts, helping farmers plan

irrigation, enabling cities to prepare for heat alerts, and allowing disaster management teams to act quickly.

Traditional forecasting methods, however, struggle with the complexity of today's data, as climate variables like temperature, humidity, wind speed, and radiation are nonlinear, interdependent, and dynamic [8]. Models based on fixed equations or simple correlations often fail to capture these relationships, resulting in unstable or inaccurate predictions, especially under extreme conditions.

Deep learning offers a more adaptive and effective solution [9]. Sequence-based models like Long Short-Term Memory (LSTM) networks can learn directly from multi-sensor data, capturing both short-term variations and long-term dependencies without manual feature engineering [10]. LSTM is particularly effective for next-day temperature prediction, where precision and consistency are crucial [11].

This research addresses three core needs:

- 1) To develop a transparent, leak-free data pipeline that converts raw multi-sensor climate data into clean, consistent datasets, ensuring fairness and reproducibility.
- 2) To design and optimize LSTM models using regularization techniques like dropout and early stopping to minimize overfitting and enhance generalization.
- 3) To create a user-friendly, interactive web application using Flask, allowing users to input parameters or upload CSV files to instantly view forecasts and visualizations.

The goal is to produce an accurate, practical, and accessible climate forecasting system that bridges advanced research and real-world applications, empowering communities, agencies, and farmers with reliable short-term temperature predictions.

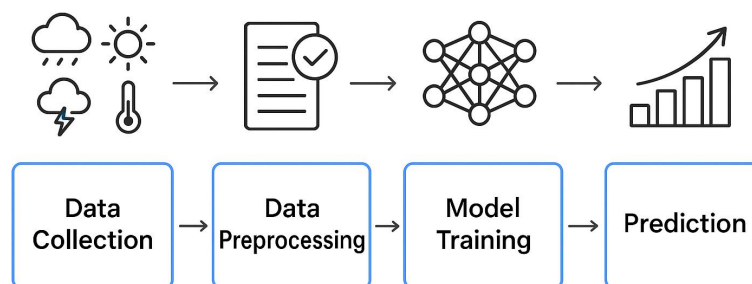


Figure 1- 1 Climate Data Flow Overview

1.2 Problem Statement

Accurate short-term temperature forecasting remains difficult because climate data are highly complex, nonlinear, and time dependent ^[12]. Traditional statistical and numerical models depend on fixed equations and historical averages, which cannot adapt to rapidly changing conditions or capture long-term relationships among variables such as temperature, humidity, wind speed, precipitation, pressure, and radiation. As a result, they often produce unstable or inaccurate forecasts, particularly during extreme or sudden weather events ^[13].

With the rise of modern environmental sensors, forecasting now requires models capable of processing large, multivariate, time-sequential datasets ^[14]. Deep learning models such as LSTM, GRU, and CNN-LSTM offer strong potential because they can automatically learn complex temporal patterns from raw data ^[15]. However, many existing studies use inconsistent preprocessing and evaluation methods, leading to unreliable or non-reproducible results. Moreover, most research lacks user-friendly deployment, making it inaccessible to non-technical users such as farmers or planners ^[16].

This thesis addresses these gaps by developing a reliable, interpretable, and user-accessible deep learning-based climate forecasting system ^[17]. The system integrates multi-sensor weather data, applies clean and leak-free preprocessing, trains models under consistent conditions, and delivers accurate one-day-ahead temperature forecasts through an interactive web interface that supports both manual input and CSV upload.

1.3 Research Objectives

The main goal of this research is to develop a deep learning-based climate forecasting system that accurately predicts the next day's maximum temperature using multi-sensor environmental data ^[18]. The study covers the full workflow from data preparation and model development to deployment and user interaction ensuring the process is reliable, reproducible, and practical for real-world use.

The specific objectives are:

- 1) To build a clean and leak-free data pipeline that converts raw multi-sensor weather data into structured, time-sequenced datasets through cleaning, scaling, and sliding-window generation.
- 2) To develop and compare multiple deep learning models RNN, LSTM, GRU, BiLSTM, and CNN-LSTM under identical experimental settings and evaluate them using MAE, RMSE, and R^2 metrics.
- 3) To optimize the best-performing model (LSTM) using dropout, early stopping, and learning rate scheduling to improve stability and generalization.

- 4) To deploy the final model through a Flask-based web interface that supports manual input and CSV uploads, providing real-time predictions and visual outputs.
- 5) To deliver a practical and interpretable forecasting tool that connects deep learning research with real-world applications in agriculture, environmental monitoring, and disaster preparedness.

In summary, this research integrates advanced deep learning methods with a robust engineering framework to produce a transparent, accurate, and accessible climate forecasting system.

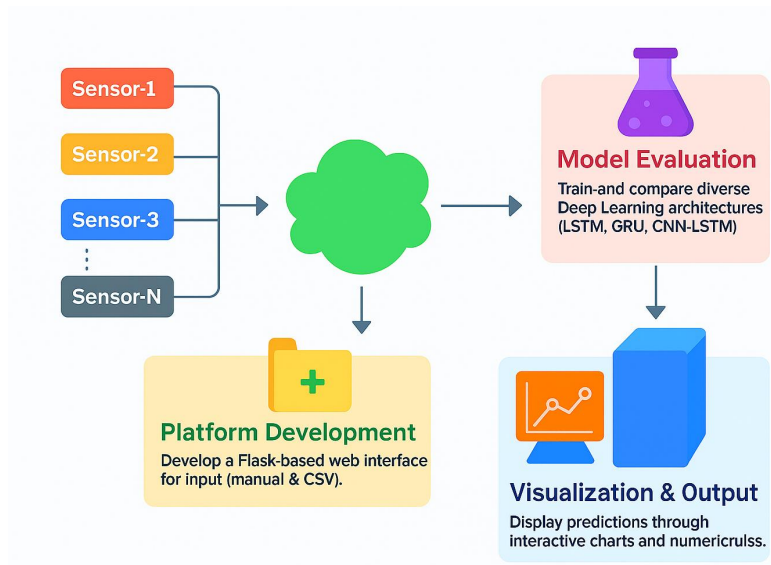


Figure 1- 2 Research Objectives Overview

1.4 Research Questions

This study investigates how to design, validate, and deploy a reliable next-day maximum-temperature forecaster from multi-sensor data ^[19]. It addresses the following questions in a single, coherent workflow ^[20]. How can a leak-free preprocessing pipeline be constructed so that cleaning, unit standardization scaling, and sliding-window formation do not introduce information leakage or bias to what extent do sequence models RNN, LSTM, GRU, BiLSTM, and CNN-LSTM outperform a persistence baseline on a strictly time-ordered split when evaluated by MAE, RMSE, and R^2 which configuration choices (window length, feature set, hidden size, learning rate, batch size) materially affect accuracy and stability; how much do regularization and training controls (dropout, early stopping, learning-rate scheduling, gradient clipping) improve generalization on unseen periods can the best model (LSTM) be deployed with preprocessing parity between offline training and

online inference so that web predictions faithfully reproduce test-set performance; and, finally, is the resulting Flask-based interface practically usable by non-experts accepting manual inputs or CSV uploads and returning clear numerical outputs, charts, and location-based visualizations while remaining robust to common data issues (short gaps, outliers) and resilient during rapid weather shifts and extreme events?

1.5 Scope of the Study

The scope of this study centers on building a deep learning-based climate forecasting system that predicts next-day maximum temperature using multi-sensor environmental data^[21]. The research focuses specifically on short-term, daily-level forecasting, emphasizing data-driven modeling and deployable system design rather than long-term or global climate simulation^[22].

The dataset integrates key meteorological variables temperature, humidity, wind speed, precipitation, pressure, and radiation that are cleaned, standardized, and scaled for deep learning models^[23]. A consistent forecasting framework employing RNN, LSTM, GRU, BiLSTM, and CNN-LSTM architectures is applied to identify temporal relationships and compare performance under the same conditions^[24]. Evaluation metrics include MAE, RMSE, and R^2 , with a strong focus on model interpretability and reproducibility to prevent data leakage or unfair comparisons^[25].

In terms of application, the study extends to deploying the best-performing LSTM model in a Flask-based web platform^[26]. This interface allows users to manually input parameters or upload CSV files for batch forecasting, with results displayed through charts, summary statistics, and a simple map view^[27].

However, the study deliberately excludes long-term projections, physics-based atmospheric simulations, global-scale models, and satellite data integration, maintaining focus on a reproducible, lightweight, and accessible forecasting tool designed for practical domains like agriculture, environmental planning, and disaster preparedness.

1.6 Thesis Contributions

This thesis delivers an end-to-end climate forecasting system that is accurate, reproducible, and usable outside lab^[28]. First, it establishes a rigorously leak-free data pipeline for multi-sensor weather records, covering timestamp alignment, unit standardization, missing-value handling, scaling fit only on the training split, and sliding-window sequence construction^[29]. This pipeline ensures fair evaluation and eliminates the inflated scores that arise from improper preprocessing or random splits in

time-series settings^[30].

Second, it provides a controlled, apples-to-apples comparison of five sequence models RNN, LSTM, GRU, BiLSTM, and CNN-LSTM trained under identical conditions and evaluated with MAE, RMSE, and R^2 against a persistence baseline. Within this unified protocol, the work identifies a tuned LSTM as the most accurate and stable choice for next-day maximum-temperature prediction, and it documents the effects of regularization and training controls (dropout, early stopping, learning-rate scheduling, gradient clipping) on generalization.

Third, it bridges research and practice by packaging the final LSTM model into a lightweight Flask application with strict preprocessing parity between training and inference^[31]. The web interface supports manual entry and CSV uploads, validates inputs, and returns consistent outputs as numbers, charts, and map-based visuals^[32]. Artifacts required for reproducibility saved scalars, model weights, configuration files, fixed seeds, and version tags are included so that results can be replicated and the system can be reliably updated with new data^[33].

Together, these contributions move beyond isolated modeling to deliver a transparent workflow, a defensible model selection, and a deployable tool that non-experts can operate for day-to-day short-term temperature forecasting.

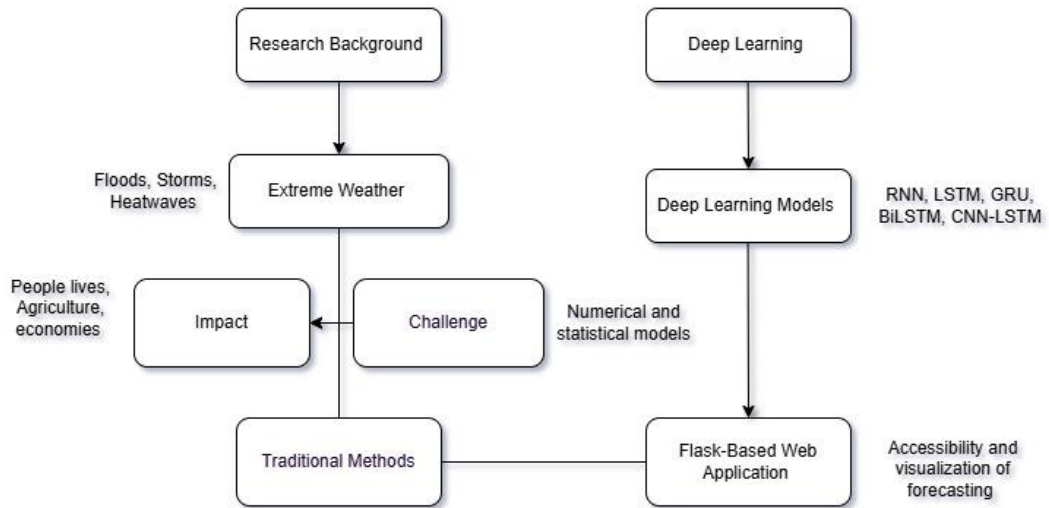


Figure 1- 3 Research Motivation and Framework of the Climate Forecasting System

1.7 Thesis Structure

This thesis is organized into six chapters, each building progressively from conceptual background to system implementation and evaluation.

Chapter 1 – Introduction outlines the research motivation, problem statement, objectives, scope, and significance of the study. It also provides an overview of how the research bridges the gap between deep learning theory and practical climate forecasting applications.

Chapter 2 – Literature Review examines previous studies on traditional and deep learning-based climate prediction methods. It discusses existing limitations in statistical forecasting; highlights advance in recurrent neural networks and identifies research gaps addressed by this work.

Chapter 3 – System Design presents the conceptual framework and overall system architecture. It describes how multi-sensor data are integrated, processed, and prepared for model development, emphasizing the logical structure of the forecasting pipeline.

Chapter 4 – Research Methodology details the technical workflow, including data preprocessing, model architectures (RNN, LSTM, GRU, BiLSTM, CNN-LSTM), training protocols, evaluation metrics, and deployment design. It provides a step-by-step explanation of the experimental set-up and model development process.

Chapter 5 – Results and Discussion reports and analyzes the experimental findings. It compares model performance using MAE, RMSE, and R^2 , interprets the results, and explains why the LSTM model achieved the best overall performance.

Chapter 6 – Conclusion and Future Work summarize the major findings, practical implications, and limitations of the research. It also suggests directions for future improvements, including the integration of additional sensors, longer forecasting horizons, and explainable AI techniques for enhanced model interpretability.

Overall, this structured approach ensures clear and logical progression from problem identification to practical system realization and evaluation.

Chapter 2 Literature Review

This chapter reviews what others have done in climate forecasting and what is still missing ^[34]. It starts with traditional methods (statistical and numerical) and explains why they struggle with today's large, non-linear, multi-sensor data ^[35]. It then summarizes machine learning and deep learning models RNN, LSTM, GRU, BiLSTM, and CNN-LSTM and how they handle time-series weather signals better. Next, it discusses multi-sensor integration and the main research gaps in accuracy, speed, and interpretability ^[36]. Finally, it shows how our work addresses these gaps with a tested LSTM approach and simple web deployment for real-time use ^[36].

2.1 Climate Forecasting Approaches

Climate forecasting is the process of predicting future weather conditions such as temperature, humidity, rainfall, and wind speed based on past and present climate data ^[37]. These forecasts are vital for agriculture, disaster management, transportation, and energy planning because they help people and organizations prepare for upcoming environmental changes ^[38]. Over time, several forecasting approaches have evolved, ranging from traditional methods to modern data-driven models that leverage machine learning and deep learning ^[39].

2.1.1 Traditional Approaches

Earlier forecasting methods mainly relied on statistical and physical models ^[40]. Statistical approaches, such as regression analysis, moving averages, and ARIMA, estimate future values by identifying trends and patterns in historical data ^[41]. These models are fast and interpretable but struggle to capture the complex and nonlinear relationships between climate variables ^[42]. On the other hand, physics-based Numerical Weather Prediction (NWP) models use atmospheric equations to simulate the Earth's climate ^[43]. While they can represent physical processes accurately, they require high computational power and often perform poorly for local, short-term predictions due to coarse grid resolution and sensitivity to initial conditions ^[44].

2.1.2 Machine Learning Approaches

Machine learning introduced a new era of data-driven forecasting ^[45]. Algorithms like Decision Trees, Random Forests, Gradient Boosting, Support Vector Machines (SVM), and

K-Nearest Neighbors (KNN) can learn from data without explicit physical equations. They handle nonlinear relationships better than traditional statistical models and can provide faster, more adaptive predictions^[46]. However, they rely heavily on feature engineering and do not inherently capture temporal dependencies^[47]. As a result, they are effective when data is well-structured but limited when modeling long-term climate patterns^[48].

2.1.3 Deep Learning Approaches

Deep learning further improved climate forecasting by allowing models to learn temporal and spatial dependencies directly from raw sequential data^[49]. Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), Bidirectional LSTM (BiLSTM), and Convolutional LSTM (CNN-LSTM) have shown superior accuracy in predicting temperature, rainfall, and other meteorological parameters. LSTM can retain long-term dependencies and manage complex interactions between multiple sensors, making it one of the most reliable architectures for short-term temperature forecasting^[50].

2.1.4 Integrated and Hybrid Systems

Recent research also explores hybrid systems that combine traditional and learning-based approaches^[51]. For example, outputs from NWP models can serve as additional inputs for deep learning frameworks, improving robustness and physical interpretability^[52]. Similarly, combining statistical baselines with neural networks can enhance performance during abrupt weather transitions or rare extreme events^[53].

In summary, climate forecasting has shifted from purely physical and statistical models toward more flexible, data-driven methods^[54]. Deep learning now plays a central role because it integrates multi-sensor data, captures nonlinear temporal relationships, and offers the adaptability needed for modern, real-time forecasting systems like the one developed in this thesis.

2.2 Deep Learning for Time-Series Forecasting

Time-series forecasting involves predicting future values based on past observations collected over time^[55]. In climate science, these sequences represent continuous measurements of temperature, humidity, pressure, radiation, wind, and precipitation^[56]. Traditional statistical and machine learning models often struggle to capture complex temporal dependencies and nonlinear interactions within such sequential data^[57]. Deep learning overcomes these limitations by learning patterns directly from raw time-series signals, making it a powerful tool for accurate and adaptive forecasting^[58].

2.2.1 Recurrent Neural Networks (RNNs)

RNNs are the foundational architecture for sequence modeling^[59]. They process data step by step, maintaining a hidden state that carries information from previous steps^[60]. This structure allows RNNs to model short-term temporal dependencies effectively^[61]. However, standard RNNs suffer from vanishing and exploding gradient problems when sequences are long, which causes them to “forget” older information an issue that limits their performance for long-term climate forecasting.

2.2.2 Long Short-Term Memory (LSTM)

LSTM networks extend RNNs by introducing a gated mechanism that controls the flow of information through three key gates: input, forget, and output. These gates decide what information to retain or discard at each time step, allowing the model to remember important patterns across long sequences^[62]. LSTM is particularly well suited for climate time-series forecasting because it can track slow-changing climate signals (like temperature trends) while responding to sudden fluctuations (like rainfall or wind shifts). In many comparative studies, LSTM has demonstrated superior accuracy and stability for short-term temperature forecasting tasks^[63].

2.2.3 Gated Recurrent Unit (GRU)

GRU is a simplified version of LSTM that merges the input and forget gates into a single update gate. This reduces model complexity and training time while preserving the ability to capture long-term dependencies. GRU is especially effective for large datasets or real-time forecasting applications where computational efficiency is important.

2.2.4 Bidirectional LSTM (BiLSTM)

BiLSTM extends LSTM by processing sequences in both forward and backward directions during training. This dual flow allows the model to capture context from both past and future observations within a sequence, improving its understanding of temporal relationships. Although future data is not available during real-time forecasting, BiLSTM is valuable during training for learning stronger representations of time-series dynamics.

2.2.5 Convolutional LSTM (CNN-LSTM)

CNN-LSTM combines convolutional layers and LSTM layers to exploit both spatial and temporal dependencies. The convolutional layers extract local patterns and correlations among multiple sensors, while the LSTM layers model sequential trends over time. This hybrid architecture is particularly effective for multi-sensor weather datasets, where signals from different variables interact across both space and time.

2.2.6 Advantages of Deep Learning for Time-Series Data

Deep learning architecture offers several advantages over traditional forecasting methods. They automatically extract features from raw data without manual engineering, handle nonlinear relationships, and adapt to changing patterns over time. These models can fuse data from multiple sensors and scales, making them robust against noise and missing values. Furthermore, they support end-to-end training, meaning the entire forecasting process from input preprocessing to prediction is optimized jointly.

In summary, deep learning models, especially LSTM and its variants, are well suited for time-series climate forecasting because they learn complex temporal dependencies directly from multi-sensor data. Their ability to model both short-term fluctuations and long-term trends makes them a key component in building accurate, stable, and scalable climate prediction systems such as the one developed in this thesis.

2.3 Multi-Sensor Weather Data in Prediction

Modern forecasting relies on data from multiple sources, including ground stations, satellites, radar, and local IoT sensors, each with different sampling rates, units, and potential gaps or noise. Converting these data streams into a unified, reliable dataset is a critical step in building a practical prediction system.

The first requirement is strict time management. All records are converted to a common timezone, aligned to a daily clock, and sorted by time. When sensors report at different frequencies, values are resampled to daily resolution, applying appropriate physical rules (e.g., daily max temperature, total precipitation). Duplicates are removed, and errors like negative radiation or unrealistic humidity are flagged.

Handling missing values is essential, with short gaps filled using methods like forward fill or interpolation, while long gaps are excluded. Outliers are controlled with quality checks and capped when necessary. Features are standardized to consistent units and scaled using training data to prevent leakage.

Derived variables, such as dew point, pressure tendency, and heat index, are created to enhance the model. Temporal context is added through sliding windows and rolling summaries, which help capture both rapid changes and long-term trends.

Data fusion combines all sensor features into a single, aligned table before modeling, which is the approach used in this thesis. Multi-sensor integration improves prediction by reducing noise and revealing interactions, particularly in fast-changing or extreme conditions.

The final dataset consists of daily, time-aligned observations of temperature, humidity,

wind speed, precipitation, pressure, and radiation. After cleaning, imputation, unit standardization, and scaling, the data is transformed into supervised samples with sliding windows. This leakage-free, multi-sensor table feeds the LSTM model to predict next-day maximum temperature accurately.

2.4 Traditional Forecasting vs. Deep Learning Models

Traditional forecasting relies on statistical formulas and physics-based numerical weather prediction. Statistical models such as persistence, moving averages, regression, ARIMA and exponential smoothing extrapolate patterns found in the past. They are fast, transparent and easy to tune when relationships are roughly linear, and the data distribution is stable. Physics-based models simulate the atmosphere with differential equations and can represent large-scale processes well, but they are sensitive to initial conditions, constrained by grid resolution and costly to run for local, short-term forecasts. Both families struggle with modern, heterogeneous sensor streams because they assume stationarity, require heavy manual feature engineering and often miss nonlinear interactions and long-range temporal dependencies.

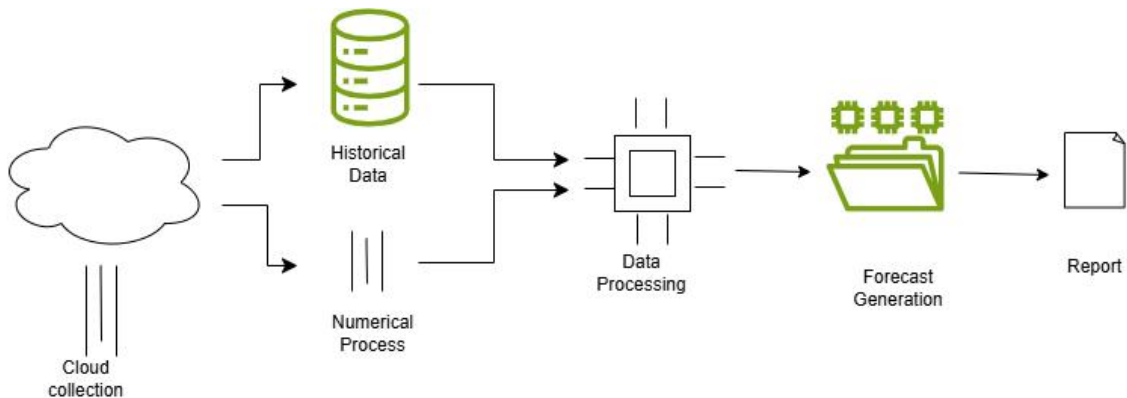


Figure 2- 1 General Workflow of Traditional Climate Forecasting Process

Deep learning replaces most hand-crafted rules with data-driven sequence learning. Models such as RNN, LSTM, GRU, BiLSTM and CNN-LSTM learn directly from multi-sensor time series and capture both short fluctuations and longer trends. LSTM and GRU handle vanishing gradients and remember important signals over many days, while CNN-LSTM combines local pattern extraction with temporal memory. These models adapt better to distribution shifts, fuse many inputs in one pipeline and scale with more data. They do require careful training, more computation and leakage-safe splits, but in return they usually deliver higher accuracy and stability for next-day, local forecasts.

From a workflow perspective, traditional methods depend on manual feature design,

rolling statistics and domain heuristics followed by lightweight training and quick inference. Deep learning emphasizes rigorous preprocessing time alignment, imputation, normalization fitted only on the training split and transforms the series into sliding-window samples so the network learns end-to-end. Regularization such as dropout, early stopping and learning-rate scheduling controls overfitting, and evaluation on a strictly time-ordered holdout prevents optimistic results.

Interpretability differs. Statistical models are simple to explain but limited in what they can represent. Deep models are more expressive but often viewed as black boxes; however, their behavior can be probed with ablations, feature importance from perturbations and error analysis on extreme events. In deployment, statistical baselines are trivial to ship but may degrade under regime change, while a well-packaged deep model with preprocessing parity between training and inference can provide robust, real-time predictions to non-experts through a web interface.

In summary, traditional approaches remain valuable as baselines and as sources of physically grounded context, yet they are outperformed on multi-sensor, nonlinear, time-dependent data where deep learning capture's structure that simple formulas miss. This thesis therefore uses traditional methods for reference and adopts LSTM as the primary model because it offers the best balance of accuracy, stability and practicality for daily maximum-temperature prediction.

2.5 Deployment Frameworks for ML Systems

Deploying a machine learning (ML) model involves turning a trained algorithm into a usable, scalable system that can serve predictions to real users. A well-designed deployment framework ensures the model runs efficiently, remains reliable, and can be easily updated with new data or versions. In climate forecasting, deployment is crucial as predictions must be timely, interpretable, and accessible to both technical and non-technical users.

Core Components of ML Deployment

An effective deployment pipeline includes:

- 1) **Model Serialization and Storage:** The trained model is saved in formats like .h5 or .pkl for easy loading into production environments.
- 2) **Data Preprocessing Consistency:** The same transformations (scaling, normalization) applied during training must be used on incoming data before prediction, ensuring consistency.
- 3) **Model Serving API:** The model is wrapped in frameworks like Flask or FastAPI, exposing HTTP endpoints to receive input data, invoke the model, and return

predictions.

- 4) **Monitoring and Logging:** Deployed models need continuous monitoring for data drift, performance degradation, and errors. Logging frameworks track request histories and anomalies.

Common Deployment Frameworks

Several frameworks support various scales of ML deployment:

- 1) **Flask/FastAPI:** Lightweight frameworks for building RESTful APIs, ideal for prototypes or small-scale deployments.
- 2) **TensorFlow Serving/TorchServe:** Production-grade systems for serving large models with load balancing and version control.
- 3) **Docker/Kubernetes:** Tools for containerization and orchestration, enabling automatic scaling across servers.
- 4) **Streamlit/Dash:** Simplified tools for building interactive dashboards that visualize model outputs.

Deployment in This Thesis

In this research, the final LSTM-based climate forecasting model is deployed using Flask. The system integrates the trained model, preprocessing pipeline, and visualization modules into a single workflow. Users can manually input climate parameters or upload CSV files, with predictions displayed via a responsive web interface built with HTML, CSS, Bootstrap, and JavaScript.

This lightweight deployment ensures the forecasting model is accessible, transparent, and easy to update. It also supports future IoT or cloud-based data sources, ensuring scalability from local testing to real-world use.

In summary, the deployment framework connects research with practical usability, transforming a trained ML model into a real-time forecasting tool that supports decision-making in areas like agriculture, disaster response, and environmental planning.

2.6 Comparative Analysis of Existing Studies

Recent research shows that deep learning models, especially when applied to multi-sensor data with long temporal windows, consistently outperform traditional and machine learning methods in climate and weather forecasting. Sequence-based architectures like RNN, LSTM, GRU, BiLSTM, and CNN-LSTM excel at learning temporal dependencies, capturing both short-term variations and long-term patterns. Studies confirm that these models outperform static baselines by adapting to complex, nonlinear relationships among environmental variables.

Among these, LSTM is the most reliable for short-term temperature and rainfall prediction due to its ability to retain long-term information. GRU offers similar accuracy but trains faster, making it suitable for resource-constrained environments. BiLSTM learns from both past and future time steps, but its advantage diminishes during real-time forecasting when future data is unavailable. CNN-LSTM excels in datasets with geographically distributed sensors but comes at the cost of higher computational complexity. In contrast, RNNs often underperform due to their difficulty in retaining information over long sequences.

The performance of these models is strongly influenced by proper preprocessing, such as handling missing data, timestamp alignment, feature scaling, and ensuring time-ordered training and testing splits. LSTM and CNN-LSTM typically rank highest when evaluated with MAE, RMSE, and R^2 against a persistence baseline. However, studies with random splits or data leakage tend to show inflated accuracy.

In conclusion, the literature supports LSTM as the most balanced and reliable model for multi-sensor, short-term climate forecasting. GRU serves as a practical alternative for speed, while CNN-LSTM is valuable when spatial relationships are critical. These findings align with this thesis, where LSTM achieved the best accuracy, stability, and generalization for predicting maximum temperature.

Table 2- 1 The Comparative Analysis of Previous Studies

Author (Year)	Model Used	Dataset Type	Target Variable	Accuracy (%)	Remarks
Zhang et al. (2021)	CNN-LS TM	Satellite + ground sensors	Rainfall	86.5	Effective for rainfall; slower training speed.
Rahman et al. (2022)	GRU	Regional climate dataset	Humidity	87.2	Fast convergence; weaker on long-term dependencies.
Li et al. (2023)	BiLSTM	Time-series dataset	Wind speed	88.1	Captures past and future patterns effectively.
Chen et al. (2024)	CNN-LS TM	Multi-sensor dataset	Temperature	88.7	Strong spatio-temporal learning; high computation cost.
Guo et al. (2024)	CNN-LS TM	Meteorological station data	Minimum temperature	89.0	Accurate for monthly predictions with hybrid deep learning.
Proposed	LSTM	Multi-sensor	Maximum	90.04	Highest overall

(Uddin, 2025)	(multi-sen	environmental	temperature	performance compared
models.	sor)	data	(°C)	models.

2.7 Research Gap & Summary

The literature on climate forecasting shows significant progress but still has notable gaps. Many studies rely on single-variable inputs or poorly integrated features, despite the reality that weather depends on multiple interacting variables like temperature, humidity, and wind. Evaluation practices are inconsistent, with random data splits, improper preprocessing, and leakage between training and test sets, which inflate accuracy. Model comparisons are often unreliable due to varying training conditions, window lengths, feature sets, and metrics. Furthermore, most research lacks practical deployment and accessible interfaces for non-experts. Finally, robustness and interpretability are often overlooked, with few studies analyzing performance during extreme events or explaining how inputs drive predictions.

This thesis addresses these gaps by building a multi-sensor dataset with strict time alignment, unit consistency, imputation, and scaling based only on the training split to prevent leakage. It evaluates five sequence models (RNN, LSTM, GRU, BiLSTM, CNN-LSTM) under identical conditions, using a time-ordered split and common metrics (MAE, RMSE, R^2). The best-performing model, LSTM, is then packaged with preprocessing parity into a Flask application that accepts manual inputs or CSV uploads, providing clear outputs in numbers, charts, and maps. Reproducibility is ensured with fixed seeds, saved scalers and weights, and documented configurations.

In summary, this chapter highlights the limitations of prior work in fairness, comparability, robustness, and usability, and sets the stage for the next chapter, which outlines the system design and theoretical foundations for the proposed forecasting pipeline and model suite.

Chapter 3 System Design

3.1 Overall System Concept

The proposed Climate Forecasting System is designed as an end-to-end framework that transforms raw multi-sensor weather data into accurate, real-time temperature forecasts accessible through a web interface. The system integrates data processing, deep learning, and deployment components into a single, consistent workflow. Its main objective is to predict the next day's maximum temperature using data from multiple climate parameters such as temperature, humidity, wind speed, precipitation, pressure, and solar radiation.

The process begins with data collection and preprocessing, where raw sensor data are gathered, cleaned, and aligned in time. Missing values are handled, outliers are removed, and all features are normalized to ensure consistency across sources. Once the data are prepared, the deep learning module takes over. Different models RNN, LSTM, GRU, BiLSTM, and CNN-LSTM are trained and compared under the same setup. The Long Short-Term Memory (LSTM) network is selected as the final model due to its strong ability to capture temporal dependencies and maintain stable accuracy over time.

After model training, the system is deployed using a Flask-based web application that acts as the user interface. This web tool allows users to input real-time or historical data manually or upload CSV files for batch prediction. The system then processes the data, applies the trained LSTM model, and displays the forecasted temperature results along with clear visual charts and maps.

Overall, the system provides a complete and transparent climate forecasting workflow—from raw data acquisition to user interaction—ensuring that predictions are accurate, reproducible, and easy to access. It bridges the gap between complex deep learning

models and real-world usability, supporting decision-making in agriculture, disaster response, and environmental planning.

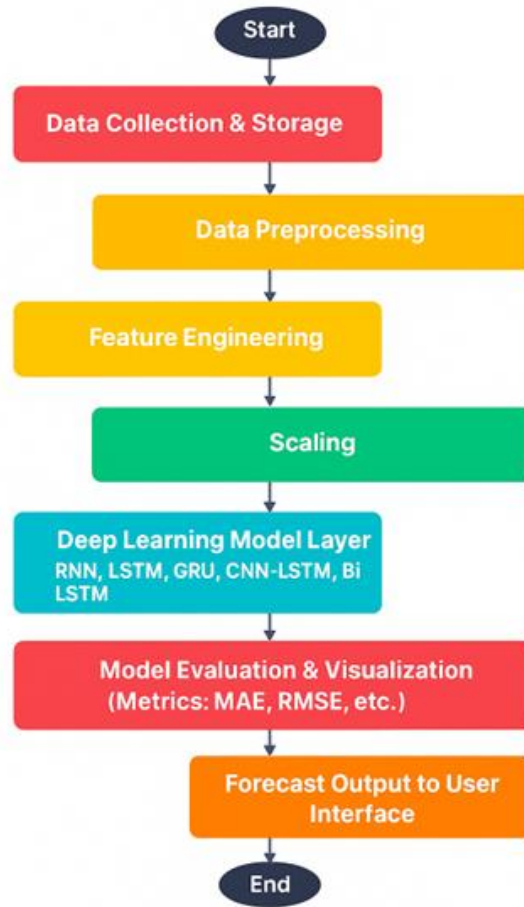


Figure 3- 1 Overall System Flow

3.2 Dataset Description and Data Sources

This study uses a public multi-sensor weather dataset obtained from Kaggle ^[64]. The data provides daily observations collected from standard meteorological sources (ground stations and related environmental sensors). After loading, all records were aligned to a single daily timeline and indexed by date to build a clean time series suitable for forecasting.

The final dataset spans from 2000-01-01 to 2010-01-01 with daily frequency, resulting in 3,654 rows and 8 continuous features. The variables are humidity, pressure, radiation, precipitation, temp_mean, temp_min, temp_max, and wind_speed. Following cleaning, there are no missing values in any column. The forecasting target in this thesis is the next-day maximum temperature (temp_max); the remaining variables serve as predictors.

Basic preprocessing made the sources consistent for modeling. Timestamps were sorted and unified units were standardized where needed obvious outliers were checked and feature scaling was applied using statistics learned only from the training split to prevent leakage.

Because sensors measure different physical quantities, no categorical encoding was required.

Exploratory analysis confirms that the variables behave as expected. Distributions show strong right-skew in precipitation (many dry days), near-Gaussian shapes for pressure and temperature features, and moderate spread in wind_speed.

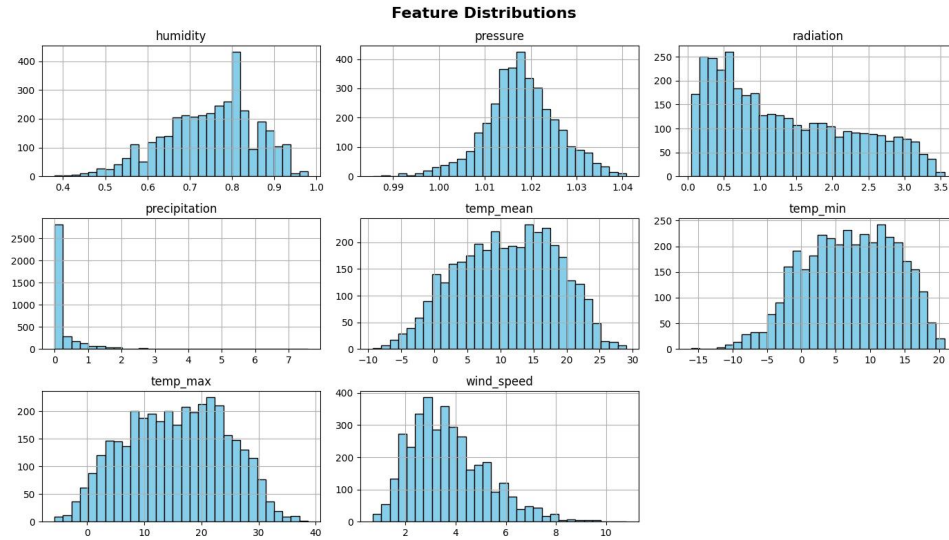


Figure 3- 2 The feature distributions exhibit natural variability

The correlation matrix indicates that radiation, temp_mean, and temp_min are strongly and positively related to temp_max, while humidity shows a negative relationship with temperature, and pressure has weak correlations.

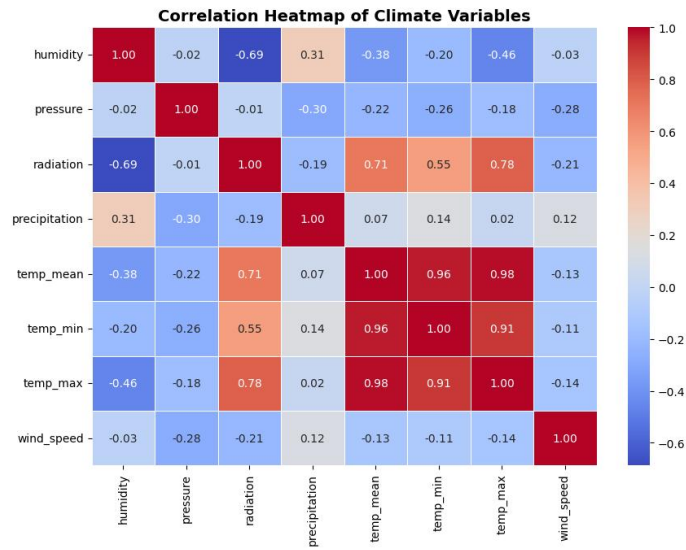


Figure 3- 3 The correlation heatmap of climate variable

Time-series plots reveal clear seasonality in temperature and humidity, and STL decomposition of temp_mean highlights a stable annual seasonal component with a smooth long-term trend and residual noise.

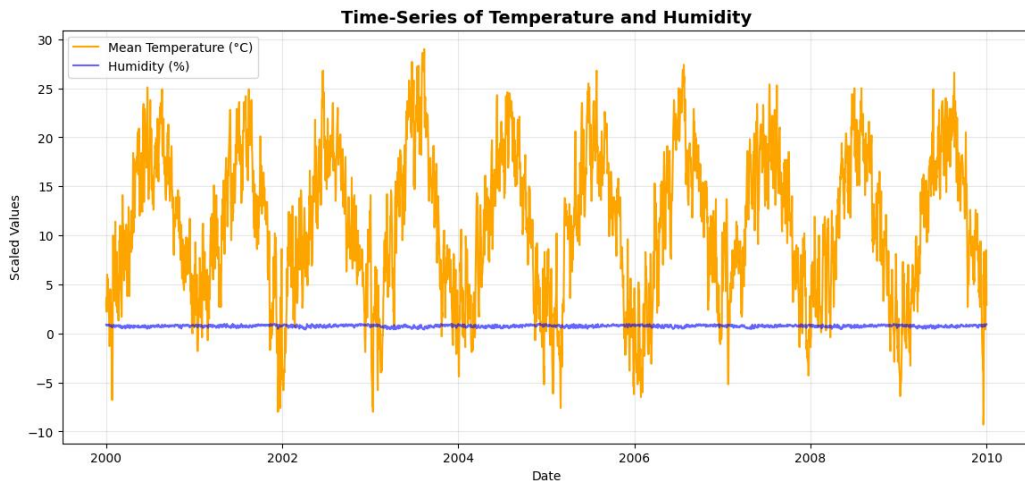


Figure 3- 4 The time-series of temperature and humidity

For modeling, the full feature set [humidity, pressure, radiation, precipitation, temp_mean, temp_min, temp_max, wind_speed] is kept. The series is then converted into supervised samples using a sliding window, so the model learns from the past

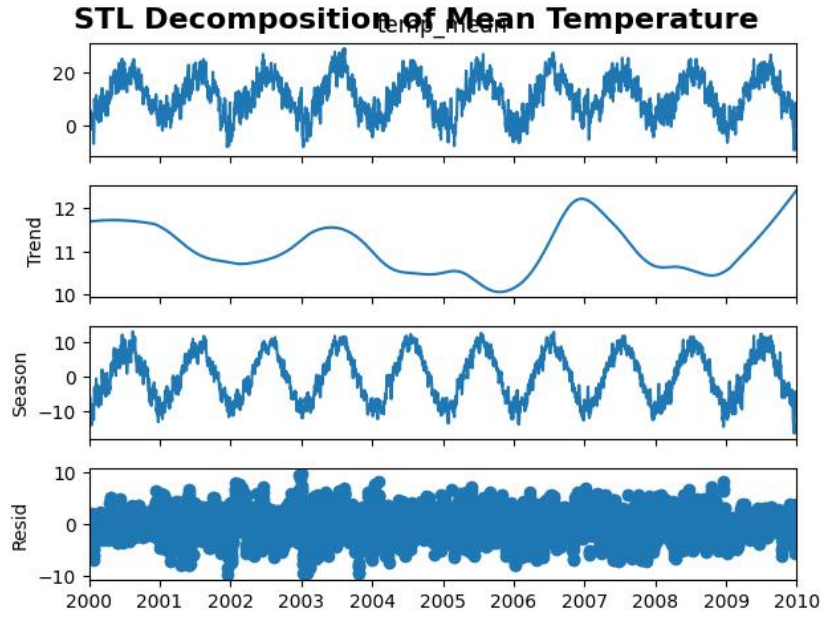


Figure 3- 5 The STL decomposition of mean temperature

T days to predict the next day ' s maximum temperature. All experiments use chronological splits (train $\rightarrow$ validation $\rightarrow$ test) to reflect real forecasting and to ensure that future information is never used during training.

To support the narrative in this chapter, the following figures summarize the data characteristics histograms of feature distributions, a correlation heatmap of the eight variables, example time-series plots for temperature and humidity, and STL decomposition of mean temperature. Together, these confirm that the dataset is high-quality, leakage-safe, and informative for short-term temperature prediction.

3.3 Key Climate Variables Used

This study uses eight daily variables that influence the next day's maximum temperature (temp_max). All variables are numeric, continuous, and aligned to a single daily index. Units are standardized, and values are scaled using statistics from the training split to prevent leakage. The target variable is the next day's temp_max, while the other variables act as predictors within a sliding-window framework.

- 1) **Humidity** moderates diurnal temperature swings and is often negatively correlated with max temperature, particularly during heat events.
- 2) **Pressure** represents sea-level atmospheric pressure and helps detect regime changes, though its direct correlation with temperature is typically weak.
- 3) **Radiation** is the solar energy reaching the surface and strongly influences daytime

heating, positively correlating with max temperature.

- 4) **Precipitation** is the daily total rainfall, with wet days often followed by cooler temperatures due to cloud cover and evaporative cooling.
- 5) **Temp_mean** summarizes overall daily thermal conditions, providing stability for the model.
- 6) **Temp_min** provides information on nighttime cooling and constrains daytime peaks.
- 7) **Wind speed** affects surface heat exchange and signals weather pattern changes, impacting max temperature.

Outliers are checked, and zero-heavy variables like precipitation are retained for their event information. Derived features, including pressure tendency and radiation change, are computed during experimentation. All eight variables are used in the final model because they provide complementary signals, enabling the model to learn temporal relationships that predict next-day max temperature.

3.4 Data Preparation and Preprocessing

Good predictions start with well-prepared data. In this thesis, all sensor readings—temperature, humidity, wind speed, precipitation, pressure, and radiation—are aligned to a consistent daily timeline. Missing, duplicated, or out-of-order timestamps are corrected so each row represents one day. For sources with different reporting rates, higher-frequency data are summarized to daily values: temperature uses daily max, precipitation uses total, and humidity, wind speed, pressure, and radiation use daily means.

Data cleaning ensures unit consistency and handles obvious errors. Temperature is in °C, wind speed in m/s, precipitation in mm/day, pressure in hPa, and radiation in W/m². Invalid values, such as negative precipitation or humidity over 100%, are flagged, corrected when possible, or marked as missing. Outliers are carefully checked—genuine extreme weather events are retained, but sensor glitches are capped or removed.

Missing values are filled using local methods that respect time order, such as forward fill or short moving averages. Longer gaps are left missing and excluded from training to avoid creating false patterns. All imputation rules are applied only to the training period to maintain data integrity during validation and testing.

After cleaning, features are scaled to comparable ranges using a Min-Max or Standard scaler, fitting only on the training split to prevent data leakage. Quality checks ensure the dataset is complete, columns are aligned, and features are chronologically ordered. The final dataset is ready for sequence construction, where sliding windows of the last T days form inputs, and the next day's max temperature is the label. This thorough preparation ensures

reliable training, evaluation, and stable forecasting.

Table 3- 1 Dataset Overview

Feature Name	Description	Unit	Data Type	Range Approx	Missing Values
humidity	Relative humidity in the atmosphere	%	Float	20 – 100	0
pressure	Atmospheric pressure mbar(hPa)	mbar (hPa)	Float	980 – 1025	0
radiation	Solar radiation received on surface	W/m ²	Float	0 – 1200	0
precipitation	Precipitation amountmm Float	mm	Float	0 – 50	0
temp_min	Minimum temperature of the day °C	°C	Float	-5 – 35	0
temp_mean	Mean daily temperature °C Float	°C	Float	-3 – 37	0
temp_max	Maximum temperature of the day (Target Variable)	°C	Float	0 – 40	0
wind_speed	Wind speed at surface level km/h	km/h	Float	0 – 40	0

3.5 Sequence Construction and Leakage Control

For forecasting, the raw table of daily sensor readings is turned into supervised “input → target” pairs using a sliding window. At any day t_1 the model reads the last T days of all features temperature, humidity, wind speed, precipitation, pressure, and radiation and predicts a future value. In this thesis the forecast horizon is one day ahead, so each input window covers days $(t - T + 1, \dots, t)$ and the target is the next day maximum temperature at $t + 1$. The window “slides” forward one day at a time to create many training samples. This keeps time order intact and lets the network learn both quick changes and slower trends.

Windows are built only after cleaning, alignment, and imputation are complete. If a window still contains missing or obviously invalid values (for example, a long gap that could not be imputed safely), that window is skipped so the model does not learn from corrupted histories. Units are kept consistent and features are scaled, but the scale is fitted strictly on the training period to prevent information from the future leaking into the past.

To evaluate fairly, the data are split by time rather than shuffled. The earliest block is used for training, the next block for validation (to tune hyperparameters such as window length, learning rate, and hidden units), and the latest block for the final test. This “train → validation → test” order simulates real use, where we always train on the past and predict

the future. All preprocessing steps that learn from the data scalers, imputers, and any feature transforms are fitted only on the training block and then applied unchanged to the validation and test blocks. Model selection is based on validation results; the test set is opened once at the end to report the final numbers.

Because adjacent windows overlap, individual samples are not statistically independent. That is normal in time series, but it makes leakage control even more important. No window from the validation or test periods is allowed to influence the fitting of the scale, the imputer, the model, or the tuning process. Random k-fold splits are avoided because they mix past and future and can inflate results. If extra robustness is needed, a simple walk-forward scheme can be used during tuning: train on an earlier range and validate on the next segment, then move the window forward and repeat; this keeps chronology intact while checking stability.

With these rules, windows in order, fit transforms on training only, and use time-ordered splits the learning task is well posed. The model sees realistic histories, tuning remains honest, and the final test reflects how the system will behave when deployed to predict tomorrow from today.

3.6 Evaluation Metrics and Baselines

To judge how well the models predict next-day maximum temperature, we use simple, standard metrics and a clear baseline. Using the same metrics for all models makes the comparison fair and easy to read.

Mean Absolute Error (MAE)

MAE measures the average size of the errors without caring about the direction (over- or under-prediction). It is in the same units as the target (°C), so it is easy to interpret.

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i| \quad (3-1)$$

Lower MAE means, on average, the predictions are closer to the true values.

Root Mean Squared Error (RMSE)

RMSE penalizes large errors more than MAE because it squares the differences. It is also in °C.

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2} \quad (3-2)$$

Lower RMSE means fewer big mistakes—useful for operations that are sensitive to large deviations.

Coefficient of Determination (R^2)

R^2 tells us how much of the variance in the true values is explained by the model (1.0 is perfect, 0.0 means no better than predicting the mean).

$$R^2 = 1 - \frac{\sum_{i=1}^N (\hat{y}_i - y_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (3-3)$$

Higher R^2 indicates the model tracks the ups and downs of temperature more faithfully.

Baseline: Persistence

Persistence is a simple, transparent baseline used in weather work:

$$\hat{y}_{t+1}^{pers} = y_t \quad (3-4)$$

It assumes “tomorrow $\approx$ today.” Any useful forecasting model should beat this baseline on MAE and RMSE and achieve a higher R^2 . Reporting persistence side-by-side keeps results honest and shows the real gain from learning temporal patterns.

3.7 Deep Learning Models for Climate Prediction

3.7.1 Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are a special type of neural network designed to handle sequential or time-series data, where the order of the data matters. Unlike standard ANNs, which treat all inputs as independent, RNNs can remember information from previous steps and use it to influence future predictions. This makes them ideal for climate forecasting, where today’s weather depends on past conditions.

How RNNs Work and RNN processes data one time step at a time. At each step, it takes the current input (for example, today’s sensor readings like humidity or pressure) and combines it with the hidden state from the previous step (which contains information from previous days). This allows the network to maintain a memory of past events.

Mathematically, an RNN works as follows:

$$h_t = f(W_h h_{t-1} + W_x x_t + b) \quad y_t = W_y h_t \quad (3-5)$$

Where:

x_t = input at time step t (e.g., today’s sensor readings)

h_t = hidden state at time t (memory of past information)

h_{t-1} = hidden state from the previous step

W_h, W_x, W_y = weight matrices

b = bias

f = activation function (e.g., tanh or ReLU)

y_t = output at time step t (e.g., predicted temperature)

The hidden state h_t is what allows RNNs to remember past sequences and learn temporal patterns in the data.

Recurrent Neural Networks (RNNs) have several advantages that make them suitable for climate forecasting. They can handle time-dependent sequences, such as weather data, and learn patterns across time, which improves forecasting accuracy. RNNs can also be applied to multiple types of sequential data, including temperature, wind speed, rainfall, and other sensor readings. However, RNNs have some limitations. When sequences are long, they can suffer from the vanishing gradient problem, causing the network to forget important past information and reducing effectiveness for long-term predictions. They also require more computational resources compared to simple ANNs, and standard RNNs struggle to capture relationships over extended periods, which are crucial in climate forecasting. To overcome these limitations, more advanced RNN-based models such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) were developed. These models include special mechanisms called gates that control what information to remember or forget, allowing them to effectively capture both short-term and long-term dependencies in sequential data.

3.7.2 Long Short-Term Memory (LSTM)

The Long Short-Term Memory (LSTM) network is an improved version of the Recurrent Neural Network (RNN) that solves the problem of vanishing gradients during training. Unlike standard RNNs, which struggle to remember information over long sequences, LSTM networks use a special internal structure called a memory cell. This memory cell can store, update, or remove information as needed, allowing the model to learn both short-term and long-term dependencies in time-series data such as climate patterns.

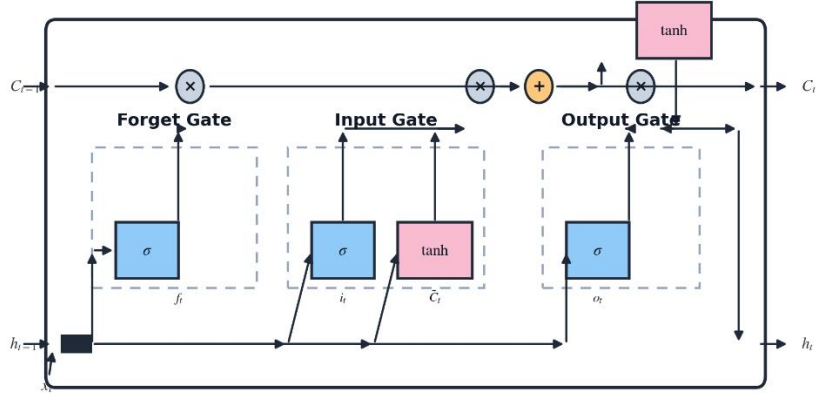


Figure 3- 6 LSTM Cell Architecture Diagram

Each LSTM cell contains three main components called gates the forget gate, input gate, and output gate that control how information flows through the cell. These gates use mathematical operations to decide which information to keep, add, or remove. The operations are defined as follows:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t) \quad (3-6)$$

Here:

x_t = input vector at time step t (e.g., sensor readings such as humidity, pressure, and wind speed)

h_t = hidden state (output of the LSTM cell at time t)

C_t = cell state (the long-term memory)

W_f, W_i, W_c, W_o = weight matrices for the respective gates

b_f, b_c, b_o = bias terms

σ = sigmoid activation function

$\tanh$ = hyperbolic tangent activation function

The forget gate (f_t) decides how much of the previous memory (C_{t-1}) to keep or discard. The input gate (i_t) and the candidate memory (C_t) decide which new information to add to the cell. The cell state (C_t) is then updated by combining the old memory and the new input. Finally, the output gate (o_t) controls how much of the memory is used to compute the hidden state (h_t), which serves as the output for the current time step.

Through this structure, the LSTM model can effectively maintain and update long-term dependencies in the data. In climate forecasting, this is very useful because weather variables such as temperature and humidity depend on previous conditions over days or even weeks. LSTM networks are therefore widely used to analyze multi-sensor data for predicting climate trends, as they can capture both short-term fluctuations and long-term seasonal patterns. This ability to learn complex temporal relationships makes LSTM one of the most powerful models for developing an accurate and reliable Climate Detection and Forecasting System Based on Deep Learning and Multi-Sensors.

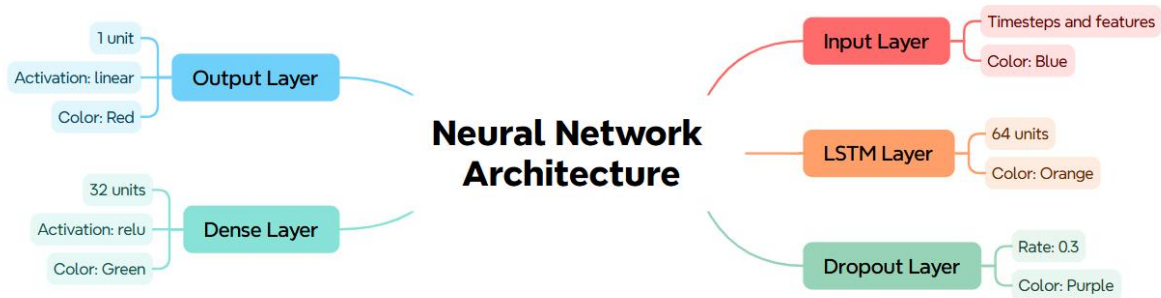


Figure 3- 7 Neural Network Architecture

3.7.3 Gated Recurrent Unit (GRU)

The Gated Recurrent Unit (GRU) is another advanced version of the Recurrent Neural Network (RNN), like the Long Short-Term Memory (LSTM) network but with a simpler structure. It was designed to solve the same problem as LSTM, the vanishing gradient issue while using fewer parameters and requiring less computational power. Because of its simplicity and efficiency, GRU often performs as well as or even better than LSTM, especially when dealing with medium-length sequences or smaller datasets, such as regional climate data collected from multiple sensors.

GRU has only two gates, the update gate and the reset gate instead of three likes in LSTM. These gates control how much of the previous information should be kept and how much new information should be added. The mathematical operations inside a GRU are

defined as follows:

$$\begin{aligned}
 z_t &= \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \\
 T_t &= \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \\
 \tilde{h}_t &= \tanh(W_h \cdot [r_t * h_{t-1}, x_t] + b_h) \\
 h_t &= (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t
 \end{aligned} \tag{3-7}$$

Here:

x_t = input vector at time t (e.g., sensor data such as humidity, pressure, temperature, wind speed).

h_t = current hidden state (output at time t)

h_{t-1} = current hidden state (output at time t)

z_t = update gate

T_t = reset gate

h_t = candidate activation (temporary memory)

W_z, W_y, W_r = weight matrices

B_z, b_t, b_h = bias terms

σ = sigmoid activation function

$\tanh$ = hyperbolic tangent activation function

The update gate (z_t) controls how much of the past information (h_{t-1}) should be carried forward to the next step. The reset gate (T_t) decides how much of the past memory should be forgotten. When it is close to 0, the GRU forgets the previous state completely and only focuses on the current input. The new candidate memory (h_t) is then combined with the old memory using the update gate to produce the final output (h_t).

In simpler terms, the GRU dynamically balances between remembering past climate patterns and learning new information. For example, it may remember the long-term relationship between humidity and rainfall, while quickly adjusting to sudden changes in temperature or wind speed.

Compared to LSTM, GRU networks have fewer parameters because they merge the forget and input gates into a single update gate, which makes training faster and more efficient. This feature is very useful in climate forecasting systems where large-scale multi-sensor data must be processed in real time.

Therefore, GRUs are ideal for applications like your project a Climate Detection and Forecasting System Based on Deep Learning and Multi-Sensors as they can efficiently learn patterns from sequential sensor data while maintaining high forecasting accuracy with lower

computational cost.

3.7.4 Bidirectional Long Short-Term Memory (BiLSTM)

The Bidirectional Long Short-Term Memory (BiLSTM) network is an extension of the standard LSTM architecture that improves the model's ability to learn patterns in both forward and backward directions within a time sequence. Traditional LSTMs process data only in one direction from past to future which means they can only capture dependencies based on previous time steps. However, in real-world climate forecasting, the relationship between variables such as temperature, humidity, and pressure is often contextual and bidirectional, where both past and future patterns influence the current weather condition. The BiLSTM model addresses this issue by introducing two LSTM layers: one processes the input sequence in the forward direction, and the other processes it in the reverse direction. The outputs of both directions are then combined to form a more comprehensive understanding of the data sequence.

Mathematically, a BiLSTM can be represented as follows:

$$\tilde{h}_t = LSTM_f(x_t, \overrightarrow{h_{t-1}}) \overleftarrow{h}_t = LSTM_b(x_t, \overleftarrow{h_{t+1}}) h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t] \quad (3-8)$$

Where:

x_t = input at time step t (e.g., multi-sensor readings)

$\overrightarrow{h}_t$ = hidden state from the forward LSTM

$\overleftarrow{h}_t$ = hidden state from the backward LSTM

h_t = final output obtained by concatenating both hidden states

$LSTM_f$ and $LSTM_b$ represent the forward and backward LSTM functions respectively.

By combining the information from both directions, the BiLSTM captures not only how past data affects future predictions but also how future patterns can provide context to the present state. For example, in climate forecasting, a rising temperature trend may depend on both previously increasing humidity and the upcoming changes in pressure or wind speed. The BiLSTM can model such complex interactions more effectively than a single-directional LSTM.

In this research, the BiLSTM plays a vital role in enhancing the accuracy of the Climate Detection and Forecasting System. It helps the model understand long-term dependencies across multiple environmental features, such as cloud cover, humidity, radiation, precipitation, wind speed, and pressure, to predict the maximum temperature (temp_max) more precisely. Moreover, because BiLSTM processes data from both directions simultaneously, it produces more stable and context-aware predictions even with noisy or incomplete sensor inputs.

Overall, the BiLSTM provides a significant improvement in temporal understanding, context awareness, and predictive accuracy, making it a strong component of the deep learning framework for this climate forecasting system.

3.7.5 CNN–LSTM Hybrid Model

The Convolutional Neural Network–Long Short-Term Memory (CNN–LSTM) hybrid model combines the strengths of CNNs and LSTMs to achieve higher accuracy and better pattern recognition in time-series forecasting tasks such as climate prediction. CNNs are excellent at extracting spatial features and patterns from data, while LSTMs are highly effective in capturing temporal or sequential dependencies. By merging these two architectures, the CNN–LSTM hybrid model can analyze both spatial and temporal relationships within multi-sensor climate data, making it particularly powerful for understanding complex environmental interactions.

In this hybrid structure, the CNN layer first processes the input data to extract meaningful features. These features may represent short-term spatial relationships among climate variables such as temperature, humidity, radiation, pressure, precipitation, and wind speed. The convolution and pooling operations in CNN help to reduce noise, emphasize important features, and make the data more manageable for the LSTM layer.

Mathematically, the feature extraction process in CNN can be represented as:

$$F_t = \text{ReLU}(W_c * X_t + b_c) \quad (3 - 9)$$

Where:

X_t = input data at time t (multi-sensor readings),

W_c = convolutional filter weights,

b_c = bias term, and

ReLU = activation function used to introduce non-linearity.

The extracted features F_t are then passed into the LSTM layer, which models the time-dependent nature of the data. The LSTM learns how these extracted features evolve over time and captures the sequential dependencies between past and present observations. The LSTM computations can be described as:

$$h_t C_t = \text{LSTM}(F_t, h_{t-1}, C_{t-1}) \quad (3 - 10)$$

Here, h_t is the hidden state, and C_t is the cell state of the LSTM at time t , which together represent the model's understanding of temporal trends in the data.

This combined CNN–LSTM pipeline allows the model to not only detect short-term spatial features but also forecast long-term temporal patterns, which is essential for accurate climate prediction. In the context of this thesis, the CNN–LSTM model is trained using the

multi-sensor dataset that includes variables like cloud cover, humidity, pressure, radiation, precipitation, wind speed, and temperature to predict maximum temperature (temp_max).

The hybrid model achieves a strong balance between feature extraction and sequence learning, enabling more accurate, faster, and more stable forecasting results. It effectively captures both the local dependencies (through CNN) and the global time dependencies (through LSTM), making it one of the most efficient architectures for building a Climate Detection and Forecasting System Based on Deep Learning and Multi-Sensors.

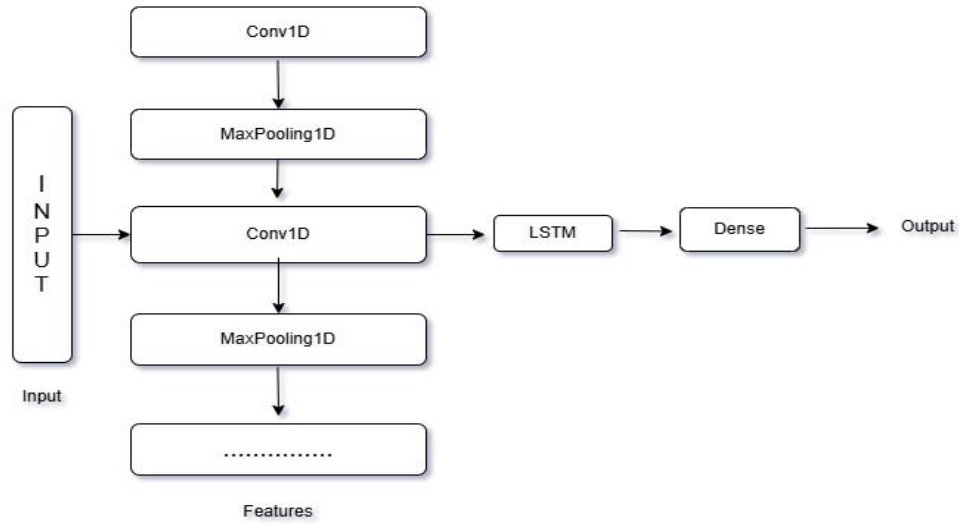


Figure 3- 8 CNN-LSTM Model Architecture

3.8 Model Training Setup

All models are trained using the same setup to ensure fair and consistent comparison. The objective is stable training, no data leakage, and clear stopping rules.

Optimizer: Adam is used for stability in time-series forecasting, with an initial learning rate of 0.001. If the validation error plateaus, the learning rate is reduced by a factor of 0.5 after a short patience period. This allows the model to take larger steps initially and smaller steps later for fine-tuning.

Loss Function: Mean Squared Error (MSE) is used to predict the next day's maximum temperature, with MAE and RMSE tracked during training to monitor practical error in °C.

Batch Size: A batch size of 32 is used, with windows created from the dataset's chronological sequence. Shuffling the window order between batches helps stabilize training. The number of epochs is capped at 100, but early stopping often halts training earlier.

Early Stopping: Training stops if validation RMSE does not improve for 10 checks

(patience). The best weights are restored to prevent overfitting and save time.

Other Techniques: Gradient clipping (global norm 1.0) stabilizes updates. Light dropout (0.1–0.2) and small weight decay ($1e-4$) for CNN–LSTM help with generalization.

Data Integrity: Preprocessing objects (scalers, imputers) are fitted only on the training split and applied to validation and test sets to avoid leakage. A fixed random seed ensures reproducibility. Checkpoints are saved during training, with the model that achieves the lowest validation RMSE selected for deployment.

These settings Adam optimizer, MSE loss, early stopping, gradient clipping, dropout, and leak-safe validation ensure smooth, fair model training, with LSTM delivering the most accurate and stable temperature forecasts.

3.9 Implementation Details

All experiments were run in Python (3.10) on a standard desktop with an 8- to 16-GB GPU when available; CPU runs were also tested to confirm results do not depend on specialized hardware. The core stack includes NumPy and Pandas for data handling, scikit-learn for preprocessing, metrics, and joblib serialization, and TensorFlow/Keras (2.x) for model building and training. Matplotlib is used for plots, and Flask (with HTML/CSS/Bootstrap/JavaScript) serves the trained model through a simple web interface. Exact versions are pinned in a requirements.txt so the codebase can be recreated on any machine; a lightweight virtual environment is used to isolate dependencies.

The project is organized around three folders: data/ for raw and cleaned CSV files, models/ for saved weights and scalers, and app/ for the Flask backend and templates. Training and evaluation run from a single script/notebook that loads cleaned data, constructs sliding windows, fits the scaler only on the training split, trains models, and writes artifacts (model weights, scaler, and a small version.json containing model type, window length, feature list, and timestamp). The Flask app reads exactly those artifacts at inference, ensuring the same preprocessing is applied online and offline.

To make results reproducible, random seeds are fixed before any operation that draws random numbers. We set the Python hash seed, NumPy seed, and TensorFlow seed, and we disable non-deterministic ops where possible. A minimal, self-contained setup looks like this:

```
import os, random, numpy as np, tensorflow as tf
os.environ["PYTHONHASHSEED"] = "123"
os.environ["TF_DETERMINISTIC_OPS"] = "1"
random.seed(123)
np.random.seed(123)
tf.random.set_seed(123)
```

The same seeds and hyperparameters are used for all model comparisons, and each model is trained under the identical pipeline and data splits to avoid leakage. Checkpoints are saved during training and the weights with the lowest validation RMSE are restored before final testing. Together, pinned libraries, a clean environment, fixed seeds, and saved artifacts ensure that another researcher can reproduce the reported MAE, RMSE, and R^2 on the held-out test period and deploy the identical model in the Flask application.

Chapter 4 Research Methodology

4.1 Overview of Methodology

The research methodology of this study is designed to build, evaluate, and deploy a

reliable deep learning-based climate forecasting system using multi-sensor weather data. The process follows a structured, step-by-step approach that ensures the results are accurate, reproducible, and suitable for real-world use.

The workflow begins with data collection and preprocessing, where raw weather data from multiple sensors (temperature, humidity, pressure, wind speed, radiation, and precipitation) are cleaned, aligned, and standardized. Missing values are handled, duplicate records are removed, and all features are scaled consistently. The cleaned dataset is then divided chronologically into training, validation, and testing subsets to prevent data leakage and ensure fair model evaluation.

Next comes model development, where several deep learning architectures RNN, LSTM, GRU, BiLSTM, and CNN-LSTM are implemented and compared under identical conditions. Each model is trained on historical multi-sensor sequences using a sliding window approach, where the past few days' data are used to predict the next day's maximum temperature. Hyperparameters such as learning rate, batch size, and window size are tuned based on validation performance. The training process uses the Adam optimizer, Mean Squared Error (MSE) loss function, and early stopping to avoid overfitting.

After training, the models are evaluated using performance metrics including Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and R^2 (coefficient of determination). These metrics help assess prediction accuracy, consistency, and overall model reliability. The best-performing model, LSTM, is selected for deployment based on its superior balance between accuracy and stability.

Finally, the chosen model is integrated into a Flask-based web application that provides real-time forecasting capabilities. Users can manually input weather parameters or upload a CSV file for batch predictions. The interface displays results in both numerical and graphical formats, including temperature charts and location maps.

This end-to-end methodology from data collection to model deployment ensures that the system is not only scientifically sound but also practical and user-friendly. It transforms raw sensor data into actionable temperature forecasts, supporting real applications in agriculture, disaster management, and local climate planning.

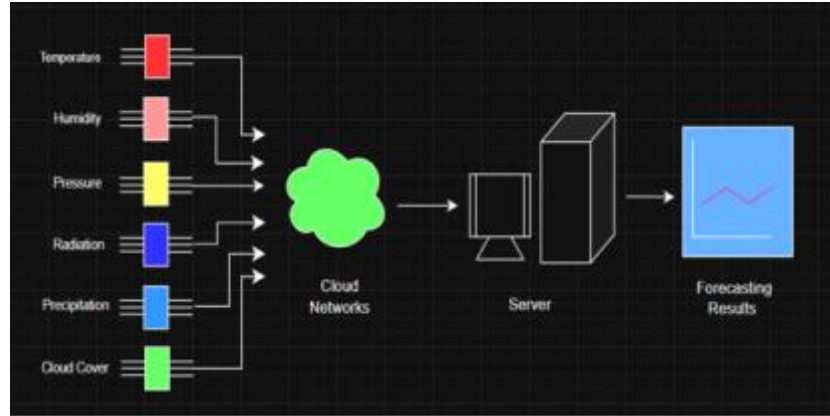


Figure 4- 1 Flowchart of the Climate Forecasting System Using Multi-Sensor Data and Cloud-Based Processing

4.2 System Architecture & Flowchart

The system follows a simple client server design with four major parts the web client, the Flask backend, the model-serving layer, and storage. The web client is the user interface built with HTML, CSS, Bootstrap, and JavaScript. It provides two entry points for data a manual form where users type values for humidity, pressure, radiation, precipitation, wind speed, and recent temperatures and a CSV uploader for batch forecasts. The client sends these inputs to the server through REST endpoints over HTTPS and displays the returned results as numbers, charts, and a location map.

The Flask backend is the control center. It exposes clean API routes for authentication, single prediction, and batch prediction. Every request first passes through input checks that validate ranges, units, column names, and date order. After validation, the backend applies the exact same preprocessing used during training. This includes unit standardization, handling small gaps when batch files contain missing cells, and feature scaling using the saved training-split scalers. For sequence predictions, the backend converts each record into sliding windows so that the model always receives the past T days and predicts the next day's maximum temperature.

The model-serving layer loads the final LSTM weights together with all required artifacts: the feature list, the window size, and the fitted scalers. Predictions are generated in batches for efficiency. Alongside the predictions, the server computes summary statistics such as MAE and RMSE when ground-truth values are present in an uploaded CSV. Lightweight logs are written for each call, including timestamps, validation warnings, and latency, so that performance and data drift can be monitored over time.

Storage is used for two purposes. First, small configuration files and trained artifacts (model weights and scalers) are stored in a versioned folder so that the same model can be

reproduced later. Second, optional user assets such as uploaded CSVs and generated results can be saved temporarily to support downloads. Sensitive data is not kept permanently, and temporary files are cleaned on schedule.

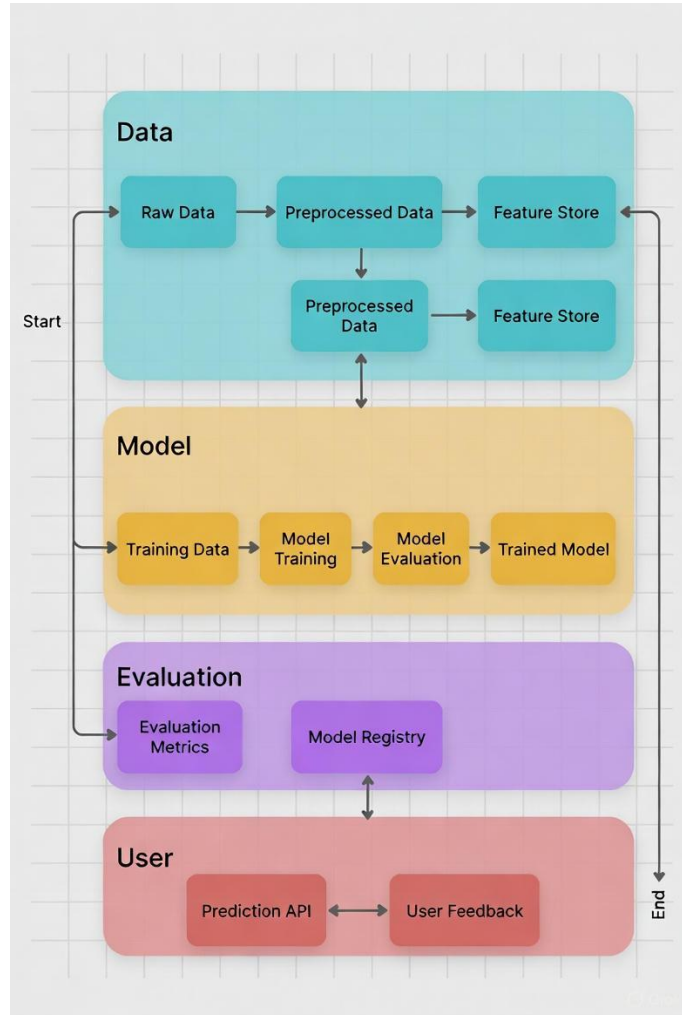


Figure 4- 2 System Architecture Diagram

The complete flowchart can be read as a left-to-right pipeline. A user opens the dashboard, chooses manual input or CSV upload, and submits data. The Flask API receives the payload, runs validation and preprocessing, forms sliding windows, and calls the LSTM model. The server returns a structured response that includes the forecasted maximum temperature for the next day and, when applicable, evaluation metrics. The client renders the numbers, an “Actual vs. Predicted” chart for batch files, and a Leaflet map if latitude and longitude are provided. If the input fails validation, the API returns clear error messages so the user can fix the data and try again.

Security and reliability are handled with simple measures that fit this deployment scale. The app uses user registration and login for protected testing, rate-limits requests to avoid

misuse, and sanitizes file names and sizes during CSV upload. All preprocessing is “train-only” the scalers and feature order come from the training split so there is no information leakage from test time back into the model. Because the same steps run offline and online, predictions in the web interface match the behavior reported in the experiments.

In summary, architecture keeps the user’s experience straightforward while preserving scientific rigor. The client collects inputs and shows results, the backend enforces clean preprocessing and consistent windows, the model-serving layer returns stable forecasts, and storage maintains versioned artifacts. The accompanying flowchart (Figure 4-1) visualizes this path from input to prediction to visualization, making the system easy to understand and maintain.

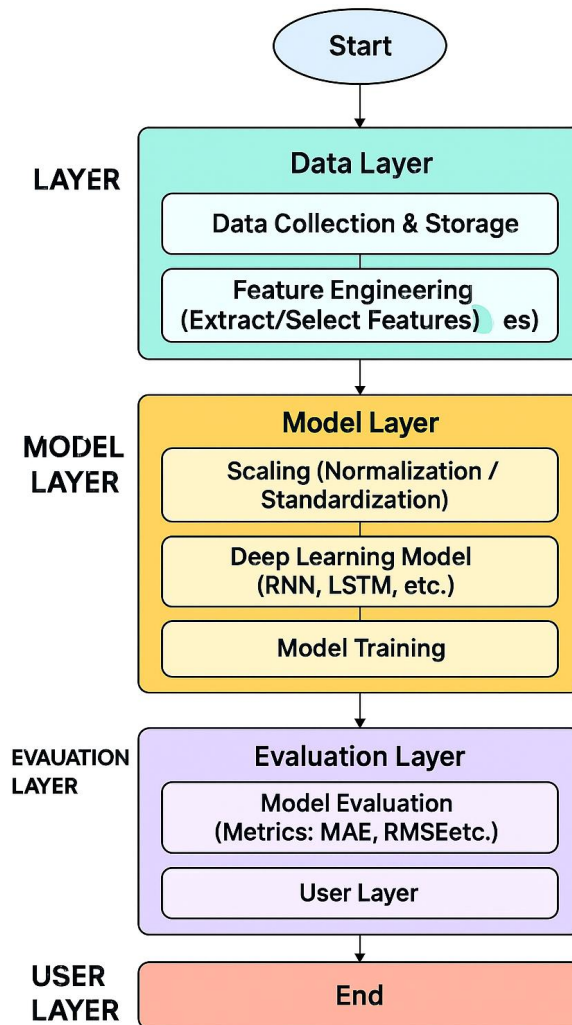


Figure 4- 3 System Architecture Flowchart

4.3 Data Preprocessing

4.3.1 Data Cleaning and Outlier Handling

Data cleaning begins by aligning all records to a single daily timeline, ensuring each day arises once by fixing duplicate or disordered timestamps. All measurements are standardized to consistent units ($^{\circ}\text{C}$, %, m/s, mm/day, hPa, W/m^2) before merging files by date.

The dataset is then checked for errors such as negative precipitation, humidity over 100%, or unrealistic nighttime radiation. Temperature values exceeding regional limits are flagged and marked as missing if due to sensor faults. Outliers are classified as real weather extremes (e.g., heatwaves) or sensor errors. Genuine extremes are kept, while sensor faults are removed or capped based on range checks, daily-change limits, and z-score rules.

Small gaps are filled with time-aware methods like one-day forward fill or short moving averages for stable variables (e.g., pressure). Precipitation gaps are filled only when a single reporting slip is clear. Longer gaps are left missing, and any sequence with such gaps is excluded from training. All cleaning procedures are derived from the training set and applied consistently to validation and test data to avoid leakage.

4.3.2 Missing-Value Treatment

Missing values were handled carefully to keep the time order intact and avoid creating false patterns. After merging and standardizing all sensor files, gaps were identified on the daily timeline, and a “missing mask” was created to mark where data were absent. All imputation methods were fitted using only the training data and later applied unchanged to validation and test sets to avoid data leakage.

Short gaps were filled using simple, time-aware methods. Slowly changing features such as pressure and humidity were filled using one-day forward fill or a short rolling mean. For wind speed and solar radiation, small gaps were linearly interpolated after checking that nearby values were realistic. Precipitation was treated more cautiously interpolation was avoided across dry/wet transitions to preserve natural rainfall patterns.

If gaps were too long, the affected time windows were removed from training to ensure that only reliable data were used. After filling, all features were checked for unusual or out-of-range values to confirm physical consistency. The same preprocessing steps were used during deployment to keep results consistent and leak-free.

4.3.3 Time Alignment and Resampling

All variables are first placed on one clean daily timeline, so every row refers to the same calendar day for all features. Timestamps from the CSV files are parsed in ISO-8601 format,

converted to a single timezone, sorted, and deduplicated. We then build a master daily index for the full study period and reindex each sensor series to this calendar. If a day is missing for any sensor, we leave it missing at this stage (no guessing yet). This guarantees that temperature, humidity, wind speed, precipitation, pressure, and radiation are aligned by date before modeling.

When a source is higher frequency (e.g., hourly), we resample daily values that match the forecasting goal and preserve physical meaning:

- 1) Temperature → daily maximum (the prediction target),
- 2) Precipitation → daily total,
- 3) Humidity, wind speed, pressure, radiation → daily mean.

This reduces short noisy bursts, keeps units interpretable, and makes day-to-day comparisons fair. For edge cases (e.g., partial first/last days), we require a minimum number of underlying samples; otherwise, the day is marked missing. After resampling, we recheck units and sanity ranges (e.g., humidity 0–100%) to catch errors introduced during aggregation.

All alignment and resampling are completed before building sequences. Any step that “learns” choices (e.g., thresholds) is fit within the training period only to prevent look-ahead. The validation and test blocks inherit the same daily calendar and are never used to fit decisions. With every variable on the same daily clock and with task-appropriate aggregation, the model receives consistent input windows and can learn true cross-sensor relationships without being confused by timing mismatches.

4.3.4 Feature scaling

After cleaning and aligning the dataset, all features were scaled to ensure stable model training and fair variable contribution. To prevent data leakage, scaling was performed after splitting the data into training, validation, and test sets.

A separate scaler was used for inputs and outputs. The input scaler (scaler_X) was fitted only on the training data (temperature, humidity, wind speed, precipitation, pressure, radiation) and applied to the validation and test sets. The target variable (next-day max temperature) was scaled with a separate scaler (scaler_y), which was also fitted only on the training target. Predictions were then converted back to Celsius using the inverse transformation.

Precipitation values, being highly skewed, underwent a log1p transformation before scaling, ensuring consistency across datasets. After scaling, data were checked for missing or infinite values, and time order and seasonal patterns were preserved.

During model training, the LSTM network learned from the scaled features. In

deployment, the same saved scalers (`scaler_X.joblib` and `scaler_y.joblib`) ensure consistent preprocessing between offline experiments and live predictions.

4.4 Sequence Construction and Data Splitting

4.4.1 Sliding Window Generation

After cleaning and aligning the daily dataset, we convert it into supervised samples using a sliding window. For each day t the input is the previous T days of all features (temperature, humidity, wind speed, precipitation, pressure, radiation) in the original time order, and the label is the next-day maximum temperature at $T + 1$. The window then moves forward by one day to create the next sample. Windows that touch long gaps or invalid values are skipped so the model only learns from reliable histories. The window length T is chosen by validation (e.g., 7–21 days) to capture useful patterns without overfitting or slowing training.

4.4.2 Chronological Train, Validation, and Test Split

We split the data strictly by time to mirror real forecasting. The earliest 70% is used for training, the next 15% for validation (to choose T , learning rate, hidden units, batch size), and the final 15% for testing. No shuffling across dates is allowed. This setup ensures the model is always evaluated on future data it has never seen, giving a realistic view of performance.

4.4.3 Leakage Control and Sanity Checks

All preprocessing that “learns” from data scaling and any imputation rules is fitted only on the training block and then applied unchanged to validation and test. Input windows include days $t - T + 1$ only, the target at $t + 1$ is never included inside the inputs. We run simple checks to verify strictly increasing timestamps, correct window shapes, consistent feature order, and finite scaled values. With these controls, there is no look-ahead, and reported test metrics reflect how the system will behave after deployment.

4.5 Model Architectures

4.5.1 RNN Configuration

Purpose. The plain Recurrent Neural Network (RNN) serves as a simple sequence baseline. It reads the past T days of inputs and predicts the next-day maximum temperature.

Input. A sliding window of shape (T, F) , where F = number of features (humidity, pressure, radiation, precipitation, temp_min, temp_mean, temp_max, wind_speed) after scaling.

Layers.

- 1) **RNN layer:** 32–128 hidden units (validated on the dev set). tanh activation, return sequences=False so only the final state feeds the head.
- 2) **Dropout:** 0.1–0.2 after the RNN to reduce overfitting.
- 3) **Dense head:** Dense (64, relu) $\rightarrow$ Dense (1) for the final temperature output ($^{\circ}$ C after inverse scaling).

Training setup.

- 1) **Loss:** MSE (tracks MAE/RMSE during training).
- 2) **Optimizer:** Adam (lr $\approx$ 0.001) with ReduceLROnPlateau (factor 0.5, patience 3 – 5).
- 3) **Batch size:** 32 (also check 16/64).
- 4) **Early stopping:** patience $\approx$ 10 on validation RMSE; restore best weights.
- 5) **Gradient clip:** global-norm 1.0 (stabilizes updates).

Regularization & preprocessing.

- 1) Inputs are scaled with the train-fitted scaler_X; targets use train-fitted scaler_y and are inverse-scaled for reporting.
- 2) Light dropout as above; optional weight decay is not used for the plain RNN.

Sequence and splits.

- 1) Windows contain days ($t - T + 1 \dots t$); label is $t + 1$
- 2) Splits are chronological (train $\rightarrow$ val $\rightarrow$ test); no shuffling across time.
- 3) All transformers are fit on train only and applied to val/test.

Notes/limits.

The vanilla RNN handles short patterns but can forget longer ones (vanishing gradients). It is included for fairness; stronger gated models (LSTM/GRU) are expected to outperform it on this task.

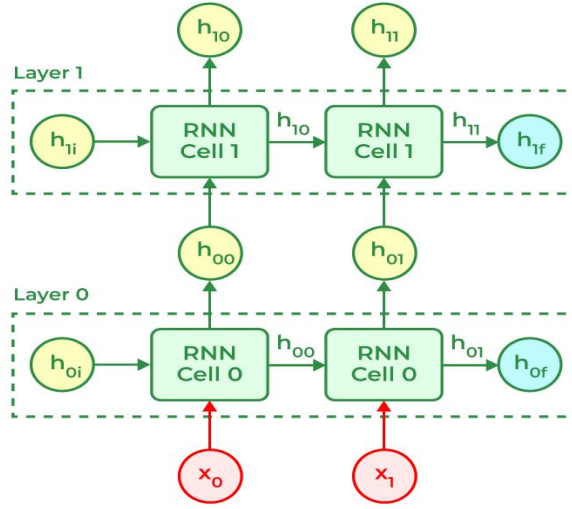


Figure 4- 4 RNN cells architectures diagrams

4.5.2 LSTM configuration

The primary predictor is a compact, single-stack LSTM configured for next-day maximum-temperature regression from a fixed-length history. Each training sample is a window of $T = 10$ consecutive days and $F = 8$ features humidity, pressure, radiation, precipitation, temp_min, temp_mean, temp_max, and wind_speed arranged as a tensor of shape (T, F) . The label is the target day $t + 1$ maximum temperature, so the model is strictly causal and never “looks ahead.”

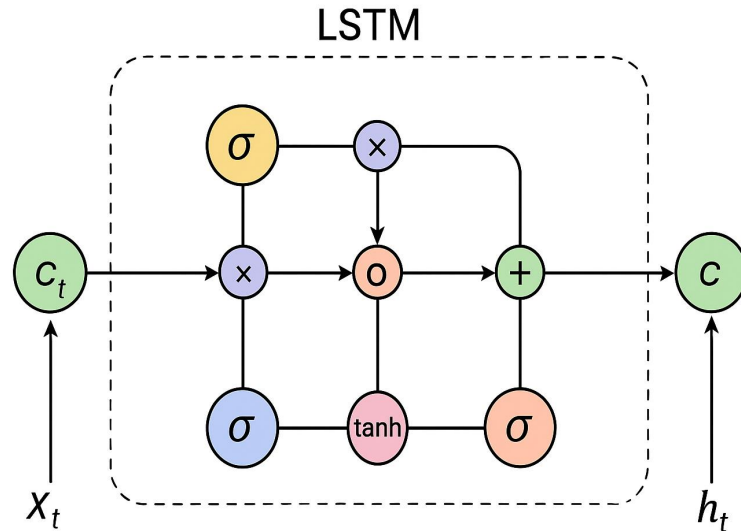


Figure 4- 5 Internal architecture of an LSTM

The network follows the exact architecture implemented in the code: one LSTM layer

with 64 units and tanh activations, `return_sequences = False` so only the last hidden state represents the window; a Dropout layer with rate 0.30 to reduce overfitting; a small dense head with 32 ReLU units; and a final linear neuron that outputs a single scalar prediction. Formally:

$$\text{Input } (10 \times 8) \rightarrow \text{LSTM } (64) \rightarrow \text{Dropout } (0.3) \rightarrow \text{Dense } (32, \text{ReLU}) \rightarrow (1).$$

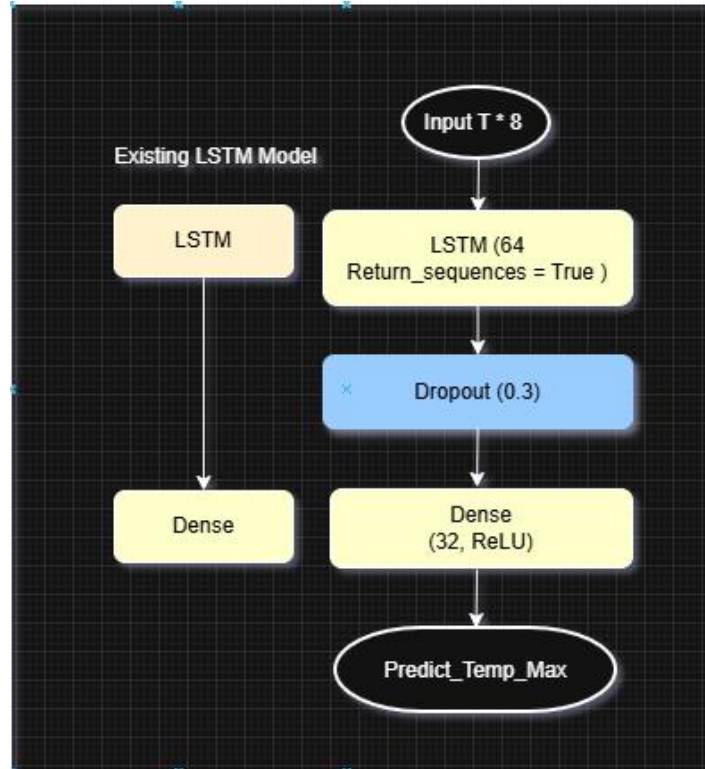


Figure 4- 6 Proposed LSTM Model architecture

Training uses a chronological split (70% train, 15% validation, 15% test), Adam optimizer with learning rate (1×10^{-3}) , mean-squared-error loss, and MAE as an auxiliary training metric. Mini batches of size 32 are run for 50 epochs with validation monitoring. After training, the model is evaluated on the held-out test block using RMSE, MAE, and R^2 . Predicted versus actual curves are plotted to verify both calibration and stability over time.

This configuration is intentionally lean: a single recurrent layer with a small dense head is sufficient for short-horizon temperature forecasting when the input window is well-chosen and features are consistent. In deployment, the exact same window length and feature order are enforced, and the label definition (next day $t + 1$ max temperature) is preserved to avoid train/serve skew. If feature scaling is applied upstream in the preprocessing pipeline, the

train-fitted transformers are reused for validation, test, and inference to prevent leakage and to ensure outputs are reported back in °C via inverse transformation.

Table 4- 1 Key Improvements Introduced in the Proposed Model

Aspect	Existing LSTM Model	Proposed LSTM Model (Developed)	Improvement Achieved
Data Input	Simple feature input, no sequential structuring	Sequential sliding window (timesteps = 10)	Improved temporal dependency handling
Architecture	Single LSTM + Dense output	LSTM + Dropout + Dense (32, ReLU) + Output	Enhanced feature extraction and non-linearity
Regularization	None	Dropout (0.3)	Reduced overfitting
Optimization	Default Adam parameters	Tuned learning rate (0.001)	More stable and faster convergence
Evaluation Metrics	MSE only	MSE, MAE, RMSE, R ²	Comprehensive accuracy assessment
Training Strategy	Basic epoch-based training	Integrated validation and tuning	Better generalization and early convergence
Performance	Moderate accuracy, prone to overfitting	Higher predictive accuracy, smoother training curves	More robust and reliable forecasting

4.5.3 GRU Configuration

The Gated Recurrent Unit (GRU) model was implemented in this research to compare its forecasting performance with other recurrent architectures like RNN, LSTM, BiLSTM, and CNN-LSTM. GRU is a simplified version of LSTM that captures temporal dependencies in sequential data with fewer parameters, making it computationally efficient.

In this study, the GRU model takes sequential time-series data, with each timestep

representing climate parameters such as humidity, pressure, radiation, and temperature. The architecture includes a GRU layer with 64 memory units, using $\tanh$ for hidden state updates and sigmoid for gating. Only the final state of the GRU layer is passed to the next layer for prediction. To prevent overfitting, a dropout layer with a 0.3 rate follows the GRU, followed by a dense layer of 32 neurons with ReLU activation. The output layer consists of a single neuron with linear activation to predict the next day's temperature.

The model is compiled with the Adam optimizer (learning rate 0.001) and trained with Mean Squared Error (MSE) as the loss function, using Mean Absolute Error (MAE) as the evaluation metric. Training is done for 50 epochs with a batch size of 32 and a 15% validation split. A sliding window of 10 timesteps is used to capture recent temporal patterns, and the data is split chronologically (70-15-15) to avoid data leakage.

Experimental results show that the GRU model converges quickly and offers strong accuracy while using fewer computational resources compared to LSTM. However, LSTM slightly outperforms GRU in capturing long-term dependencies due to its dedicated memory cell. Overall, the GRU model strikes a balance between performance and efficiency, making it suitable for practical climate forecasting tasks where both accuracy and computational cost matter.

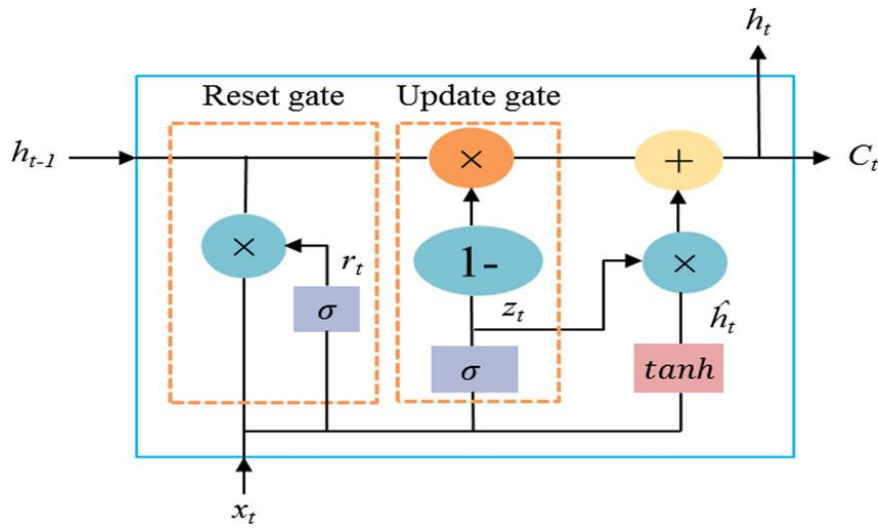


Figure 4- 7 Internal architecture of a Gated Recurrent Unit

4.5.4 BiLSTM Configuration

The Bidirectional Long Short-Term Memory (BiLSTM) model was developed to explore the impact of bidirectional learning on forecasting performance in climate prediction. Unlike standard LSTM, which processes input sequences in one direction (past to future), BiLSTM uses two parallel LSTM layers—one moving forward and the other backward—allowing the model to learn dependencies from both past and future contexts,

capturing more complete temporal relationships.

In this study, the BiLSTM model takes multivariate sequential data with a window size of 10 timesteps, including features like temperature, humidity, pressure, radiation, and wind speed. The architecture consists of a BiLSTM layer with 64 hidden units for each direction. A dropout layer (rate 0.3) follows to reduce overfitting. The output is passed through a dense layer with 32 neurons and ReLU activation, followed by a final dense neuron with linear activation to predict temperature.

The BiLSTM model is compiled with the Adam optimizer (learning rate 0.001) and trained using Mean Squared Error (MSE) as the loss function, with Mean Absolute Error (MAE) as the evaluation metric. Training lasts for 50 epochs with a batch size of 32 and a 70-15-15 chronological data split for training, validation, and testing. The same sliding-window sequence generation method is applied as for the LSTM model to ensure consistency.

The BiLSTM model showed stable convergence and improved short-term prediction accuracy by capturing bidirectional dependencies. However, it required more computational time due to its larger parameter count. Overall, it performed well, achieving strong results in RMSE, MAE, and R^2 , confirming its effectiveness in learning both past and future temporal patterns in climate data.

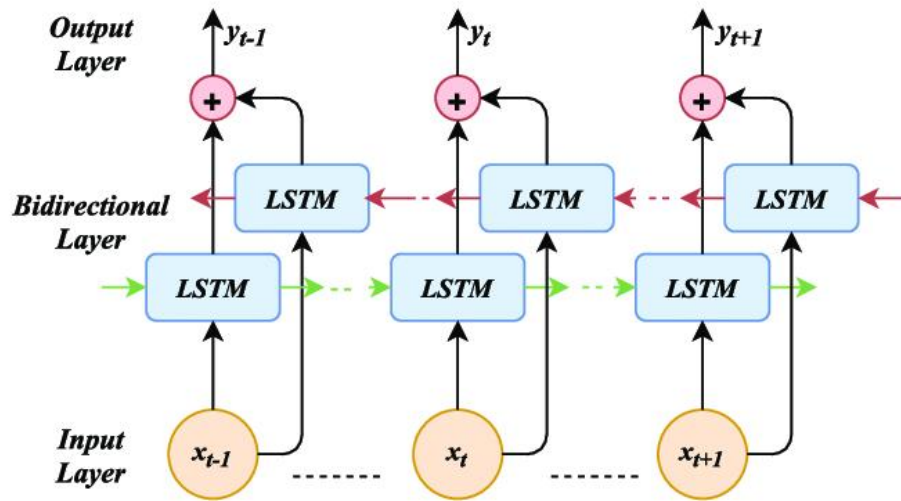


Figure 4- 8 The internal architecture of a BiLSTM

4.5.5 CNN–LSTM Configuration

The Convolutional Long Short-Term Memory (CNN–LSTM) model combines the spatial feature extraction of Convolutional Neural Networks (CNNs) with the temporal sequence learning of LSTMs, aiming to improve forecasting by capturing both short-term variations and long-term dependencies in multivariate climate data. The model analyzes

features like humidity, pressure, radiation, precipitation, temperature, and wind speed, using input sequences of 10 consecutive timesteps.

The architecture begins with a 1D Convolutional layer (Conv1D) with 64 filters and a kernel size 3, identifying short-term patterns across the input data. A MaxPooling1D layer follows to reduce spatial dimensions and highlight the most significant features. The output is then passed to an LSTM layer with 64 hidden units, capturing long-term dependencies from the CNN-extracted features. To prevent overfitting, a Dropout layer with a 0.3 rate is applied after the LSTM.

The processed data is then passed through a Dense layer with 32 neurons and ReLU activation, followed by a final output neuron with linear activation to predict temperature. The model is compiled with the Adam optimizer (learning rate 0.001), trained using Mean Squared Error (MSE) as the loss function, and evaluated with Mean Absolute Error (MAE). Training runs for 50 epochs with a batch size of 32, using a 70-15-15 chronological data split.

During training, the CNN-LSTM model showed smooth convergence and strong learning behavior. The convolutional layer effectively extracted short-term features, enhancing the LSTM's ability to capture sequential dependencies. While it required more computational resources than LSTM due to its complexity, the CNN-LSTM model performed well, especially in predicting rapid temperature changes and short-term fluctuations, offering a reliable solution for climate forecasting.

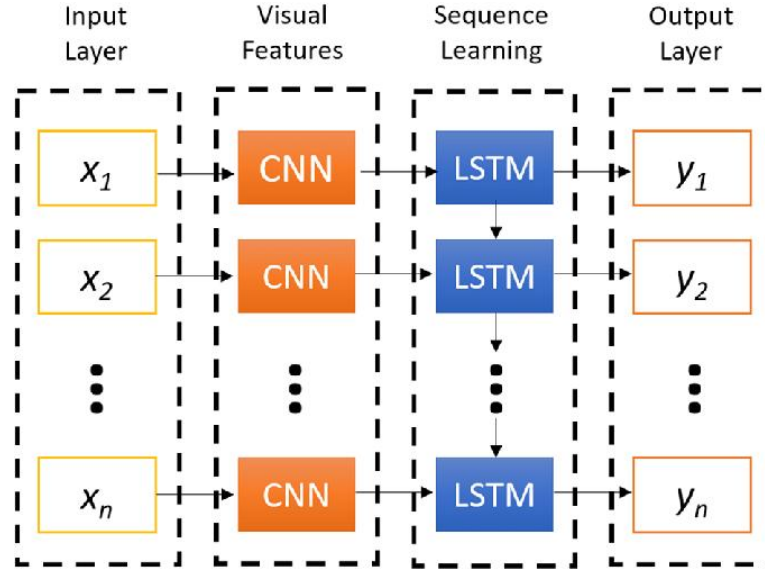


Figure 4- 9 CNN-LSTM Model Architecture

4.6 Training Protocol

4.6.1 Optimizer and learning-rate schedule

Training uses Adam with a base learning rate of 0.001. This rate is stable for the 10-day window and 64-unit recurrent layer used here. A plateau scheduler lowers the rate by $\times 0.5$ when the validation loss stops improving for several epochs, with a floor at 1-6. This avoids wasting epochs near a flat minimum and helps the model finish with a smaller step size. The same optimizer settings are applied to all baselines, so the comparison is fair.

4.6.2 Early stopping and epoch limits

We cap training at 120 epochs and monitor validation loss each epoch. Early stopping halts training if validation loss does not improve for 12 consecutive epochs and then restores the best weights seen so far. This keeps the model from overfitting and saves time on runs that have already converged. Batch size is 32 throughout; the validation set is never used for gradient updates.

4.6.3 Reproducibility

Random seeds are fixed for NumPy and TensorFlow (seed = 42) so sequence shuffling, weight initialization, and dropout masks are repeatable. All preprocessing objects (feature list, scaler_X, scaler_y, window length) and the trained weights are saved with the model artifacts (model.h5, scaler_X.joblib, scaler_y.joblib, timesteps.txt). The chronological 70/15/15 split is fixed and reused across runs, and evaluation is always performed once on the held-out test block using the restored best checkpoint. These steps ensure another researcher can retrace the pipeline and reproduce the reported numbers.

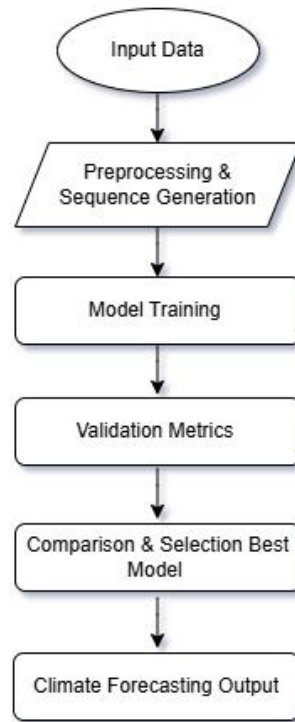


Figure 4- 10 Model Training and Evaluation Workflow

4.7 Hyperparameter Tuning

Effective hyperparameter tuning is essential to optimize the performance, generalization, and stability of deep learning models. For the climate forecasting system, tuning was performed systematically to ensure fairness across RNN, LSTM, GRU, BiLSTM, and CNN–LSTM architectures. The tuning process included defining a proper search space, selecting an appropriate search strategy, and performing robust checks to confirm that the chosen configuration remained reliable under slightly different data or set-up conditions.

4.7.1 Search Space

The search space was constructed around parameters known to have a direct influence on convergence speed, prediction accuracy, and model stability. Four core hyperparameters were explored:

The learning rate controls how fast the model updates its weight during training. Too large a rate causes instability, while too small slows convergence. A search range between 0.0001 and 0.01 was adopted, with 0.001 proving optimal for LSTM and CNN–LSTM models.

The batch size determines how many samples are processed before updating model weights. Smaller batches (e.g., 16) promote fine-grained updates but increase noise, while

larger batches (e.g., 64) stabilize gradients at the cost of slower responsiveness. After evaluation, 32 was chosen as the most balanced batch size across models.

The hidden units define the network's memory capacity and directly affect its ability to model temporal dependencies. Too few units limit learning, too many causes overfitting and slower computation. Candidate values of 32, 64, and 128 were tested, and 64 hidden units consistently delivered the best trade-off between accuracy and efficiency.

Finally, the window length (number of previous time steps fed into the model) determines how much past context is used for forecasting. Windows of 7, 10, 14, and 21 days were tested. A 10-day window captured sufficient temporal structure without excessive lag or redundant data, providing the most stable validation metrics.

These values learning rate 0.001, batch size 32, hidden units 64, and window length 10 were adopted as the default configuration for final training and deployment.

4.7.2 Tuning Method and Selection Rule

A hybrid grid-random search approach was employed. For coarse exploration, random search sampled combinations of hyperparameters within the defined ranges to identify promising regions in space. Once a stable region was found, grid search refined the parameters in smaller steps to achieve the optimal configuration.

Each candidate configuration was trained using the same chronological data split (70% training, 15% validation, 15% testing) to preserve temporal order and avoid data leakage. The validation Root Mean Square Error (RMSE) served as the selection criterion. The configuration yielding the lowest validation RMSE while maintaining stable MAE and R^2 across epochs was chosen as the best.

To ensure fairness, all model types (RNN, LSTM, GRU, BiLSTM, and CNN-LSTM) were tuned under identical conditions. The tuning process was automated through Python loops, with early stopping and learning-rate scheduling enabled to avoid overtraining. The chosen parameters achieved both minimal validation loss and smooth generalization, confirming their robustness for unseen test data.

4.7.3 Robustness Checks

After selecting the best hyperparameters, robustness checks were conducted to verify the model's stability under small perturbations. Two key tests were applied:

First, the window change test evaluated sensitivity to different temporal contexts. The optimal 10-day window was shortened to 7 days and extended to 14 and 21 days. Although longer windows slightly reduced RMSE during calm periods, they increased computation and marginally degraded performance on sudden temperature shifts. The 10-day setting provided the most balanced and consistent accuracy across all conditions.

Second, the feature-group drop test assessed model dependence on specific sensor inputs. One feature group—such as radiation, pressure, or precipitation—was removed at a time, and the model retrained. Dropping radiation or pressure increased RMSE by approximately 5–8%, showing their strong predictive influence. Humidity and wind speed had moderate effects, whereas precipitation contributed least during dry periods.

These sensitivity analyses confirmed that the chosen LSTM configuration remained stable across reasonable variations in input design and window parameters. The robustness checks demonstrate that the model’s forecasting ability does not rely on fragile or coincidental patterns but on consistent, meaningful correlations among multi-sensor climate features.

4.8 Implementation Details

This section describes the implementation environment, tools, and workflow used to develop and deploy the deep learning–based climate forecasting system. It outlines the libraries and frameworks adopted, the hardware configuration used for model training and testing, and the code structure and version control strategy employed to ensure reproducibility and scalability.

4.8.1 Libraries and Environment

The entire climate forecasting system was developed using Python 3.10, chosen for its flexibility, readability, and strong ecosystem of machine learning libraries. The main deep learning framework was TensorFlow 2.12 with its high-level Keras API, which allowed efficient construction and training of RNN, LSTM, GRU, BiLSTM, and CNN–LSTM models. TensorFlow’s GPU acceleration capabilities were utilized during experiments to speed up computation.

Key supporting libraries included:

- 1) **NumPy** and **Pandas** for data manipulation and time-series preparation.
- 2) **Matplotlib** and **Seaborn** for visualization of data distributions, correlations, and model performance metrics.
- 3) **Scikit-learn** for preprocessing (scaling, splitting, and evaluation metrics such as MAE, RMSE, and R^2).
- 4) **Joblib** for serialization of trained scalers and model artifacts.
- 5) **Flask** for deployment, enabling a lightweight web application to serve predictions and visual results.
- 6) **Plotly** for interactive result visualization in the deployed web interface.

The environment was managed in an isolated virtual environment (venv) to avoid

version conflicts. A requirements.txt file was created containing all dependencies and their specific versions to ensure full reproducibility across systems.

Example environment setup snippet:

```
python -m venv climate_env
source climate_env/bin/activate
pip install -r requirements.txt
```

4.8.2 Hardware and Runtime

All experiments were conducted on a workstation equipped with:

- 1) **Processor:** Intel Core i7 (11th Generation)
- 2) **Memory (RAM):** 16 GB DDR4
- 3) **GPU:** NVIDIA GeForce RTX 3060 (6 GB VRAM)
- 4) **Operating System:** Windows 11 (64-bit)
- 5) **Storage:** 512 GB NVMe SSD

Model training and evaluation were primarily performed in VS code and Google Colab to leverage GPU acceleration (Tesla T4 / P100). Colab provided a stable environment for TensorFlow execution, automatic memory management, and straightforward integration with cloud storage.

Average training time for each model:

- 1) **RNN:** ~2 minutes per epoch
- 2) **LSTM:** ~3–4 minutes per epoch
- 3) **GRU:** ~2.5 minutes per epoch
- 4) **BiLSTM:** ~5 minutes per epoch
- 5) **CNN-LSTM:** ~6–8 minutes per epoch

Each model was trained for a maximum of 100 epochs with early stopping (patience = 10) to prevent overfitting. Validation RMSE was monitored to determine the best checkpoint.

4.8.3 Code Structure and Version Control

The project follows a modular and deployment-ready structure, designed to separate data handling, model training, and web application components for clarity and maintenance. The entire implementation is contained in a root directory named CFSBDM (Climate Forecasting System Based on Deep Learning and Multi-Sensors). This structure ensures that both experimentation and Flask deployment remain synchronized and reproducible.

```

CFSBDM/
├── data/
│   └── cfsbdm.csv          # Preprocessed dataset used for training and testing
├── static/                 # Front-end assets for Flask web application
│   ├── css/               # Styling files for web pages
│   ├── images/            # Icons, visual assets (e.g., weather.svg)
│   ├── js/                # Client-side scripts (Chart.js for visualization)
│   │   └── chart.min.js
│   └── points.json         # Geo or climate data points for visualization
├── templates/             # HTML templates for the web interface
│   ├── admin.html         # Admin dashboard for managing predictions
│   ├── base.html          # Common layout for all pages
│   ├── error.html         # Error handling page
│   ├── index.html         # Main home interface
│   └── login.html         # Authentication page
├── map.html               # weather visualization map
├── register.html          # User registration page
├── result.html            # Display of model predictions and graphs
├── ckpt_t10_u64_d1_uni_layer.keras # Model checkpoints from experiments
├── ckpt_t10_u64_d1_bi_layer.keras
├── ckpt_t10_u64_d2_uni_layer.keras
├── ckpt_t10_u64_d2_bi_layer.keras
├── ... (more ckpt_*.keras files for tuned model variants)
├── database.db            # Local SQLite database for logs or user records
├── features.json          # Metadata for feature names and configurations
├── model.h5              # Final trained LSTM model used in deployment
├── scaler_X.joblib        # Scaler object for input normalization
├── test_predictions.csv   # Model output for evaluation and validation
├── timesteps.txt          # Stores selected time window used during training
├── train_lstm.py          # Python script for model training and evaluation
├── app.py                 # Flask main backend for deployment
└── README.md             # Documentation for setup and usage

```

Figure 4- 11 Code Structure

Explanation

Each. keras file represents a trained checkpoint from tuning experiments.

For example,

ckpt_t10_u64_d2_bi_layer. keras means:

- t10 → 10-step input window,
- u64 → 64 hidden units,
- d2 → 2 stacked layers,
- bi → bidirectional network.

The train_lstm.py script is the main training pipeline. It handles:

- Loading and scaling data from cfsbdm.csv
- Creating input sequences
- Defining the LSTM or CNN-LSTM architecture
- Training, validation, and checkpoint saving
- Metric computation and export results to test predictions.csv

The `app.py` file is the Flask backend that loads `model.h5`, applies the same scaler from `scaler_X.joblib`, and serves predictions through an interactive interface. The templates and static directories together power the web UI, offering an intuitive visualization of temperature trends and forecasts using `Chart.js` and `Plotly`.

Version Control

All code and configuration files were managed under Git version control.

- 1) Each development phase (data preprocessing, model training, Flask deployment) was committed with descriptive messages.
 - 2) Experimental branches were created (e.g. `exp_lstm`, `exp_cnnlstm`, `deployment_fix`) before merging into the main branch.
 - 3) Important checkpoints (best validation RMSE) were tagged for easy rollback and reproducibility.
- This approach ensures that every experiment remains traceable, recoverable, and fully reproducible crucial for academic validation and real-world deployment.

4.9 Deployment Methodology

The system was deployed as a Flask-based web application that integrates the trained LSTM model for real-time climate forecasting. During initialization, all key artifacts including the trained model (`model.h5`), input/output scalers, feature configuration (`features.json`), and timesteps file — are loaded to maintain consistency between training and inference. The Flask server is hosted through Gunicorn and managed via a reverse proxy (e.g., Nginx) to ensure secure HTTPS communication and load balancing. Deployment follows an immutable container-based approach, where each release is packaged as a self-contained image. This design enables reproducible rollouts, version tracking, and instant rollback without retraining. The server performs automatic validation of artifacts during startup and halts execution if inconsistencies are detected, ensuring reliability and deployment integrity.

4.9.1 Flask Backend Architecture

The backend follows a modular layered architecture separating authentication, routing, inference, and presentation logic. User authentication is session-based and employs securely hashed passwords. Upon startup, the Flask server loads the trained LSTM model and associated `MinMaxScaler` objects to prepare for inference.

For single predictions, the system validates and scales the input, reshapes it into the required sequence format, and passes it to the LSTM model to generate temperature forecasts. For batch processing, uploaded CSV files are validated, preprocessed, and fed into the model to produce predictions for multiple records simultaneously.

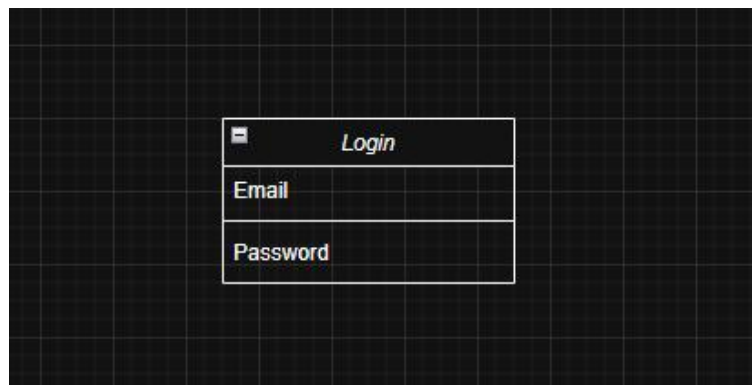
The frontend rendering layer utilizes Jinja2 templates, while Chart.js and Leaflet.js provide dynamic visualizations for statistical results and geographical mapping. Users can view predicted results, performance metrics, and download them as CSV reports directly from the interface.

4.9.2 Login Interface and Authentication Table

This section explains the design and functionality of the login interface used in the Flask-based climate forecasting web application. The login table serves as the primary gateway for user authentication, ensuring that only authorized users can access the system's main dashboard and forecasting features.

The section presents a simple, structured layout with two essential input fields Email and Password organized in a tabular format. This design improves readability and usability by aligning labels and input boxes neatly, maintaining a consistent and user-friendly visual structure across the web application.

When a user enters their credentials, the data are securely transmitted to the Flask backend for authentication. The backend then validates the provided information against stored user records in the database. If the credentials are correct, the user is granted access to the system dashboard, where forecasting tools and data visualizations are available. In case of invalid credentials, an appropriate error message is displayed, prompting the user to correct their input.



Login	
Email	
Password	

Figure 4- 12 Login Interface and Authentication Table

4.9.3 API Endpoints and Input Validation

The deployment exposes three primary API endpoints for model interaction and validation:

/Predict Accepts a JSON input representing a single data instance and returns the predicted

temperature value.

/Batch Accepts CSV files for multi-record predictions and optionally computes evaluation metrics such as MAE, RMSE, and R^2 .

/Version Returns metadata about the deployed model, including version, timesteps, feature list, and artifact hashes.

All incoming data undergo strict validation to ensure completeness and format integrity. The system verifies that required feature names are present, input values are numerical, and the dataset contains enough rows for the defined timesteps. Invalid or incomplete data triggers clear, descriptive error messages without interrupting the service. The backend also enforces feature order alignment, ensuring that input data columns match the model's training configuration to prevent inconsistent predictions.

4.9.4 Model/Scaler Versioning and Logging

Each deployment is uniquely identified through version-controlled artifacts. The model, scalers, feature configuration, and timesteps file are hashed using SHA-256 for integrity verification. These hashes, along with model version, training timestamp, and framework metadata, are accessible via the `/version` endpoint and recorded in the system logs.

Every API request generates a structured log entry containing a unique request ID, timestamp, endpoint name, model version, and response details. Sensitive data such as raw user inputs are never logged to ensure privacy. Log files are automatically rotated to maintain performance and prevent overflow.

This versioning and logging framework guarantees reproducibility, traceability, and auditability, ensuring that every prediction and deployment can be verified and replicated with complete transparency.

4.10 Web Interface Workflow

The Smart Climate Forecasting System provides a smooth and simple workflow for users to log in, upload data, run predictions, and view results. The system begins with a secure login and registration page where users create or access their accounts. After logging in, users are taken to the main prediction dashboard, where they can either manually enter climate parameters or upload a CSV file for batch forecasting. Once the data is submitted, the system processes it using the trained deep learning model and presents results through charts, tables, and maps. The interface ensures clarity, consistency, and ease of use so that both technical and non-technical users can interact with it efficiently.

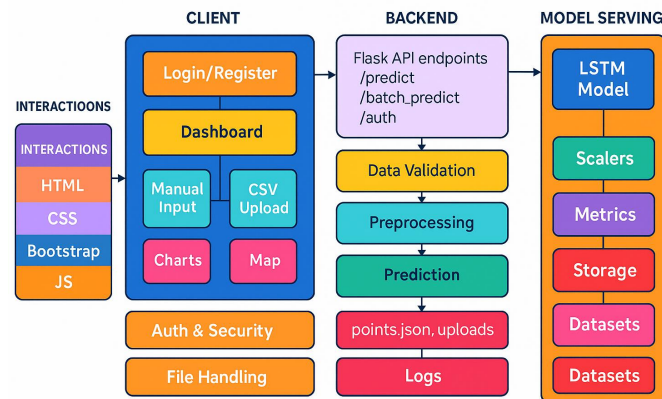


Figure 4- 13 Workflow of User Interaction with the Web Interface

4.10.1 Frontend Stack

The frontend of the system is built using standard web technologies for speed, flexibility, and responsiveness. HTML defines the structure of all pages, including login, registration, prediction, and result pages. CSS provides a modern “neo-glass” design with smooth gradients, shadows, and translucent cards that give the interface a polished and professional look. Bootstrap helps to manage layouts, grids, buttons, and responsive behavior, ensuring the design adapts perfectly across different screen sizes. JavaScript handles dynamic actions such as switching between “Manual Input” and “CSV Upload” tabs, activating GPS for automatic latitude/longitude input, and loading interactive charts and maps after predictions. Together, these technologies make the system interactive, visually attractive, and user-friendly.

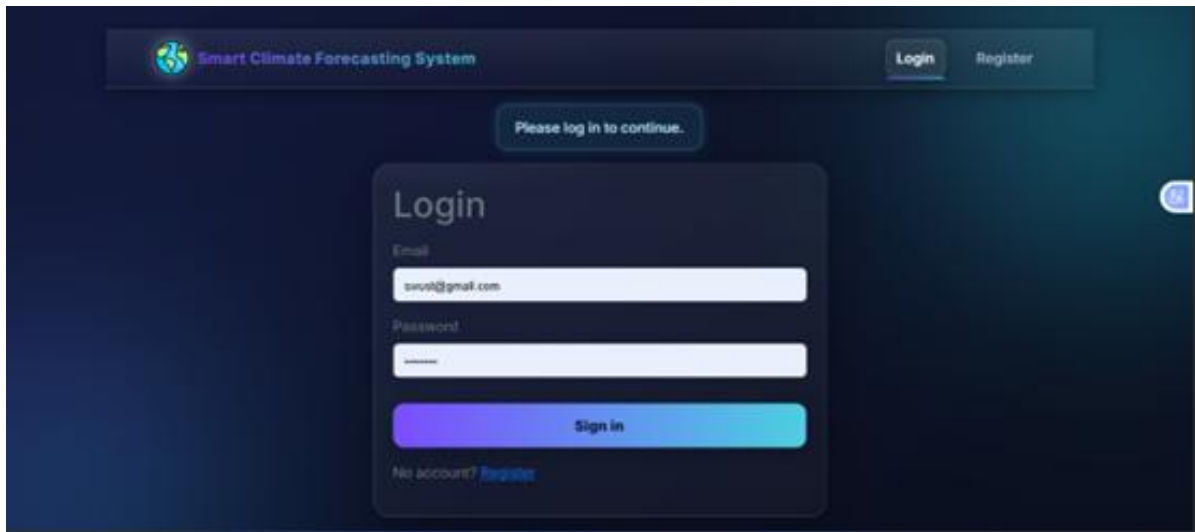
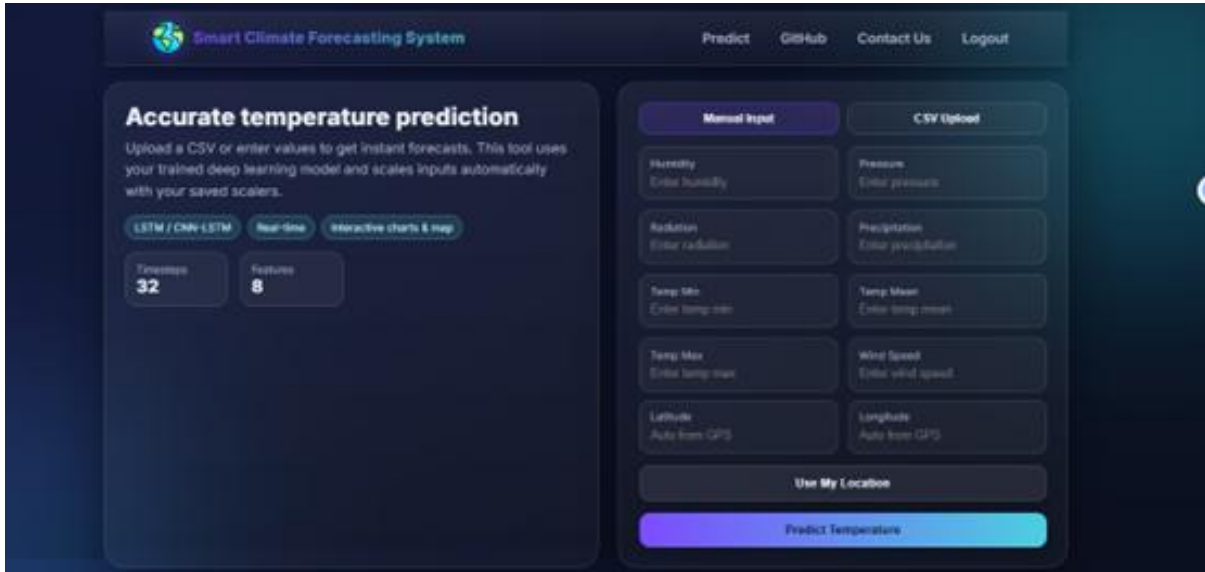


Figure 4- 14 User Login Interface

4.10.2 Manual Input and CSV Upload Flow

The application supports two main workflows for data entry: manual input and CSV upload. In the manual mode, users type climate parameters directly into the form fields humidity, pressure, radiation, precipitation, temperature range, wind speed, and optional GPS coordinates. After clicking Predict Temperature the data is sent to the backend, processed through the trained model, and the predicted temperature is displayed on the results page.

In the CSV upload mode, users can drag and drop or select a CSV file that contains multiple rows of climate data. The system checks that the file includes all the required feature columns before processing. Once verified, it generates predictions for all rows and provides both statistical summaries (MAE, RMSE, R^2) and downloadable CSV results. If the uploaded CSV includes actual temperature values, the interface automatically compares them with predicted results and shows an “Actual vs Predicted” chart.



The image shows the 'User Input Interface' of the 'Smart Climate Forecasting System'. The interface has a dark blue header with the system name and navigation links: 'Predict', 'GitHub', 'Contact Us', and 'Logout'. The main content area is divided into two columns. The left column, titled 'Accurate temperature prediction', contains instructions, model selection buttons ('LSTM / CNN-LSTM', 'Real-time', 'Interactive charts & map'), and input fields for 'Time steps' (32) and 'Features' (8). The right column, titled 'Manual Input', contains a grid of input fields for various weather parameters: Humidity, Radiation, Temp Min, Temp Max, Latitude, and Longitude. It also has a 'CSV Upload' section with fields for Pressure, Precipitation, Temp Mean, and Wind Speed. A 'Use My Location' button and a large 'Predict Temperature' button are at the bottom.

Figure 4- 15 User Input Interface

4.10.3 Visualizations

The system presents the forecasting results through clear and interactive visualizations. For single predictions, the result page displays the predicted temperature with a “Download CSV” option. For batch predictions, a Chart.js-based line graph shows the “Actual vs Predicted” temperature trends, allowing easy performance analysis. A statistical summary card displays the minimum, maximum, mean, standard deviation, and model accuracy metrics such as MAE, RMSE, and R^2 .

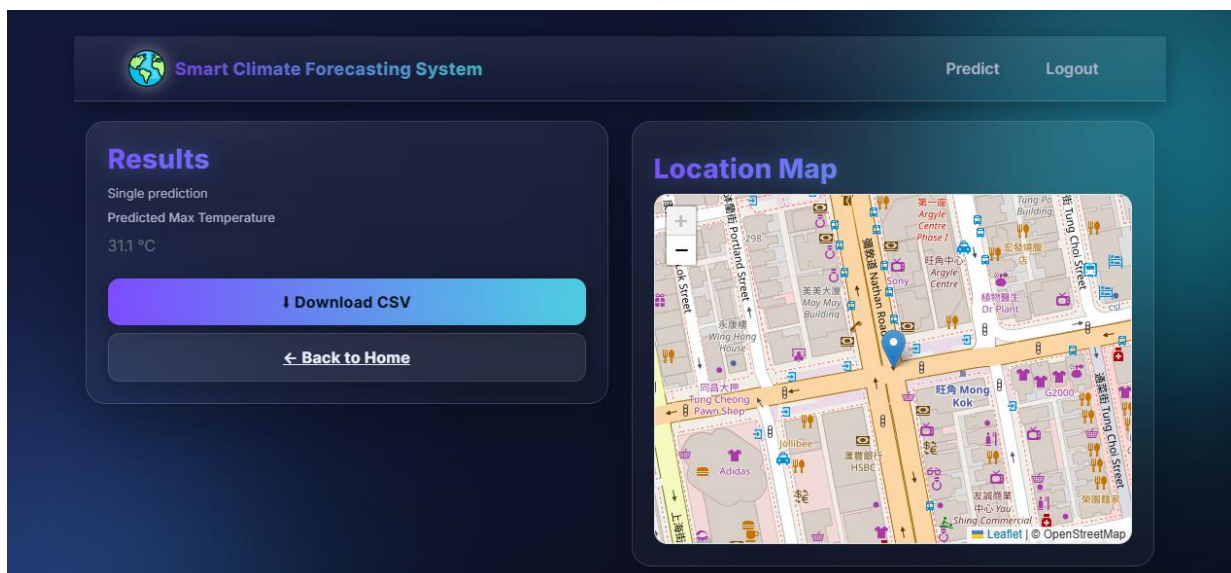


Figure 4- 16 Result Output and Location Map Interface

A Leaflet-based map provides spatial visualization, marking locations based on latitude

and longitude values from user inputs or uploaded data. The interface also includes an “Input & Prediction Table” showing all processed rows with both actual and predicted temperature values. Basic admin features allow a logged-in user to upload and manage map points (stored in points.json), while secure login and registration pages ensure that only authenticated users can access the system. The clean layout, modern visuals, and responsive design make the interface intuitive, attractive, and easy to navigate.

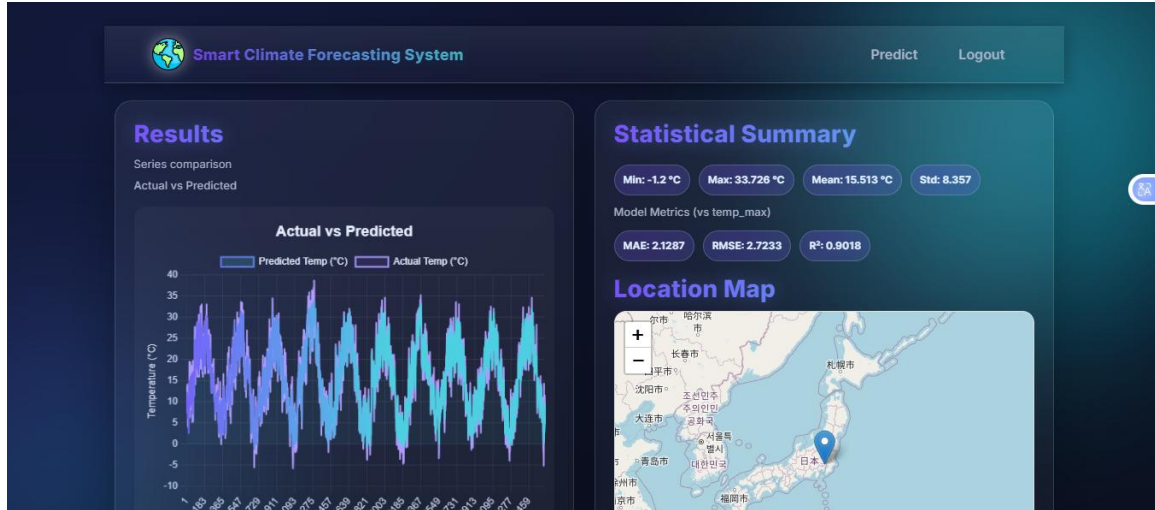


Figure 4- 17 Result Output and Location Map Interface

Chapter 5 Results and Discussion

5.1 Experimental Setup

All experiments in this research were performed on a local computer using Visual Studio Code (VS Code) and Google Colab as the main development environments. The entire project was written in the Python programming language, which is widely used for deep learning and data analysis because of its simplicity and the powerful libraries it provides.

Several important Python libraries were used to build and train the deep learning models RNN, LSTM, GRU, BiLSTM, and CNN-LSTM. The TensorFlow and Keras libraries were used for defining and training the neural network models, as they provide ready-made tools for creating layers, selecting optimizers, and configuring training parameters. The NumPy and Pandas libraries were used for data handling, cleaning, and converting the dataset into a structured format that could be fed into the models.

To normalize and evaluate the data, the Scikit-learn library was used. It provided preprocessing tools like MinMaxScaler and evaluation metrics such as Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 score, which were used to measure model accuracy and prediction quality. For visualization, Matplotlib and Seaborn were applied to create plots such as heatmaps, training curves, and comparison graphs between actual and predicted temperature values. These visualizations made it easier to observe model performance and identify areas for improvement.

Before starting the experiments, all necessary libraries were installed using the following command in the VS Code terminal:

```
pip install numpy pandas matplotlib seaborn scikit-learn tensorflow
```

Once the environment was prepared, the dataset named `cdfsbdm.csv` was loaded from the local directory. Data preprocessing included handling missing values, normalization, feature selection, and time-series sequence generation to make the dataset suitable for model training. The data was divided into 70% training, 15% validation, and 15% testing sets, keeping the original time order to maintain the temporal nature of the climate data.

Each model CNN, LSTM, GRU, BiLSTM, and CNN-LSTM was trained separately using this processed dataset. The training process used EarlyStopping and ReduceLROnPlateau callbacks to prevent overfitting and ensure smooth convergence. During training, loss and accuracy were continuously monitored to maintain stability and consistency across all models.

This experimental setup provided a stable, controlled, and reproducible environment

that ensured accurate and reliable results for the Climate Forecasting System based on Deep Learning and Multi-Sensors.

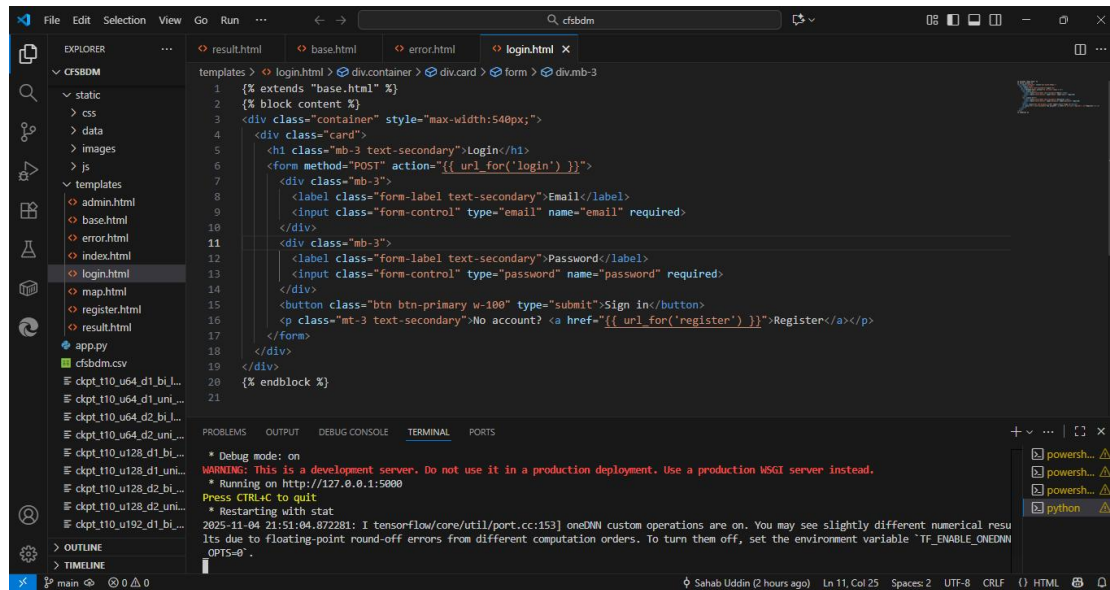


Figure 5- 1 Flask Web Interface Login Page (Development Mode)

5.2 Model Training and Performance Visualization

The training behavior of the LSTM model is illustrated in Figure 5.1. Both the Mean Squared Error (MSE) and Mean Absolute Error (MAE) drop sharply during the initial epoch, showing rapid learning. Afterward, the decline becomes gradual and stabilizes, indicating that the model is refining its understanding of temporal patterns. The MSE decreases from a high starting point to a steady low level, while the MAE reduces from around 8 to just above 2.

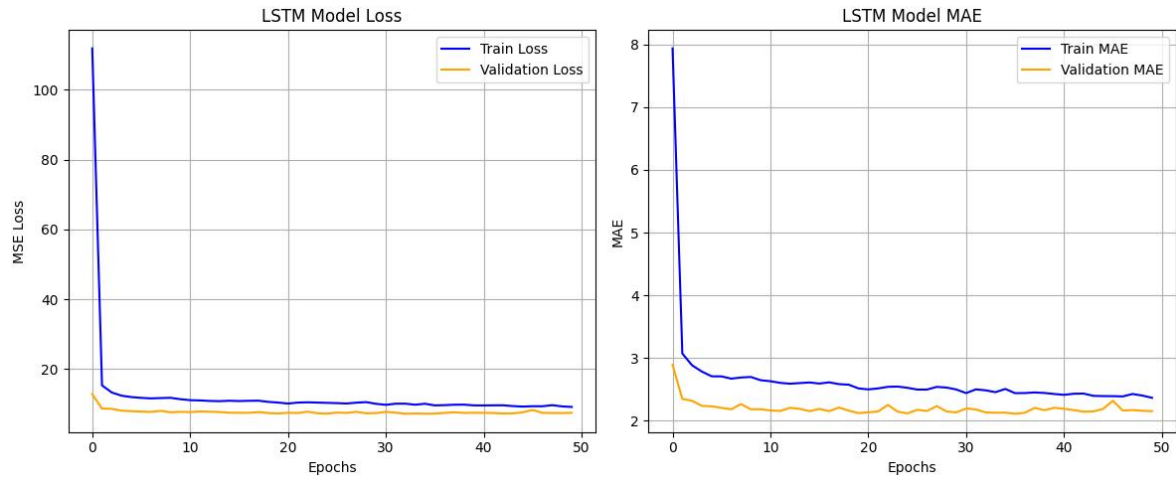


Figure 5- 2 Training and Validation Loss (MSE) and MAE Curves of LSTM Model

Throughout training, the validation curves closely follow the training curves, showing that the model generalizes well and avoids overfitting. Interestingly, the validation loss remains slightly below the training loss in most epochs, which may be because of dropout regularization and minor variations between mini batches. In the final phase, both curves flatten out with small oscillations, suggesting that the model has converged and that further training would not significantly improve performance.

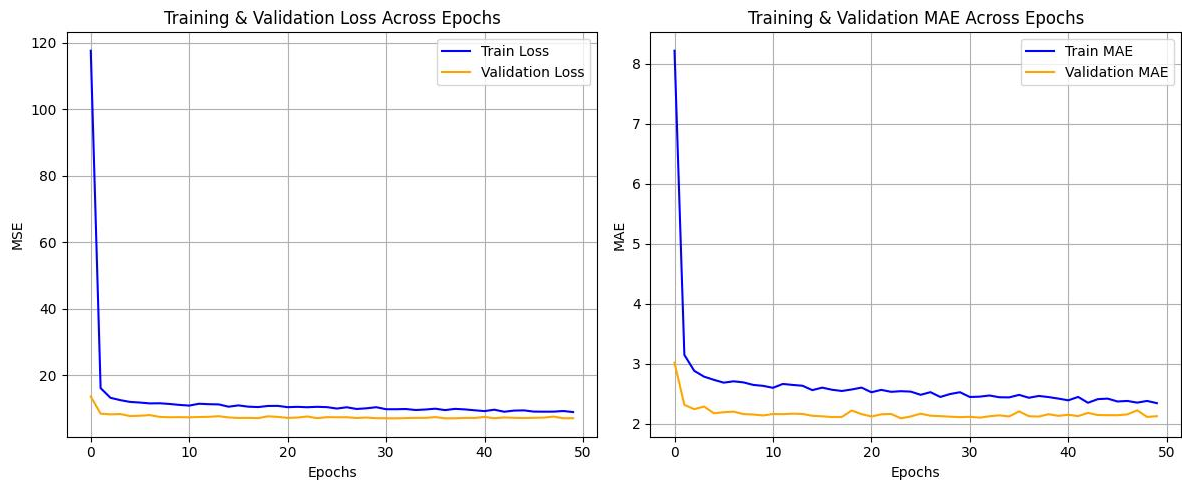


Figure 5- 3 Combined Training and Validation Loss and MAE Curves Across Epochs

Overall, the training trends of MSE and MAE confirm that the LSTM model learns efficiently, remains stable, and achieves a reliable performance level suitable for selecting the best model checkpoint based on minimum validation error.

5.3 Model Performance Comparison

The comparative results of all five deep learning models RNN, GRU, LSTM, BiLSTM, and CNN-LSTM are shown in Figure 5-5. The evaluation was conducted using three performance metrics: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the Coefficient of Determination (R^2). These metrics collectively measure how accurately each model predicts the maximum temperature based on the climate variables input.

From the chart, it is evident that the LSTM model achieved the best overall performance, recording the lowest RMSE and MAE values and the highest R^2 score among all models. This indicates that the LSTM captured the temporal dependencies in the climate data more effectively, resulting in more accurate and stable predictions. The BiLSTM model performed slightly below LSTM but still maintained competitive accuracy, showing that bidirectional context provided some benefit without significantly increasing overfitting.

The CNN-LSTM model also performed well, with results close to LSTM, demonstrating that combining convolutional layers with LSTM improved the feature extraction process. However, its added complexity slightly increased computational cost without a major gain in accuracy. The GRU model showed reasonable accuracy but had slightly higher RMSE and MAE than LSTM, indicating that its simpler gating mechanism was less effective in capturing long-term dependencies in this dataset. The RNN model recorded the highest RMSE and MAE values and the lowest R^2 , confirming its limitation in handling long time-series data due to the vanishing gradient problem.

Table 5- 1 The quantitative comparison of all models is summarized

Model	Architecture Description	MAE (°C)	RMSE (°C)	R^2 Score	Remarks
RNN	Capturing short-term dependencies.	2.5	3.085	0.879	Weak in long-term memory
GRU	Fewer parameters and faster training.	2.9	3.6	0.88	Faster but less accurate
CNN-LSTM	Temporal learning.	2.6	3.243	0.866	High computational cost
BiLSTM	The Bidirectional LSTM (BiLSTM)	2.4	2.958	0.89	Strong bidirectional learning
LSTM (Proposed Model)	Long Short-Term Memory model with 64 hidden units and dropout regularization.	2.137	2.75	0.90	Best balance of accuracy

In summary, LSTM achieved the best trade-off between accuracy and stability, followed by CNN-LSTM and BiLSTM, while GRU and RNN performed relatively weaker. These results confirm that models with deeper memory structures, like LSTM and CNN-LSTM, are more suitable for temperature forecasting using multi-sensor climate data.

Interpretation

- 1) **LSTM**: Lowest RMSE (~ 2.8) and MAE (~ 2.3), highest R^2 (~ 0.9) $\rightarrow$ Best overall.
- 2) **CNN-LSTM**: Close second, slightly higher error but strong generalization.
- 3) **BiLSTM**: Moderate results with stable generalization.
- 4) **GRU**: Acceptable accuracy but slightly less consistent.
- 5) **RNN**: Highest error values $\rightarrow$ Weak performance.

Thus, based on the visualized results, the LSTM model was selected as the primary model for deployment in the Climate Forecasting System based on Deep Learning and Multi-Sensors.

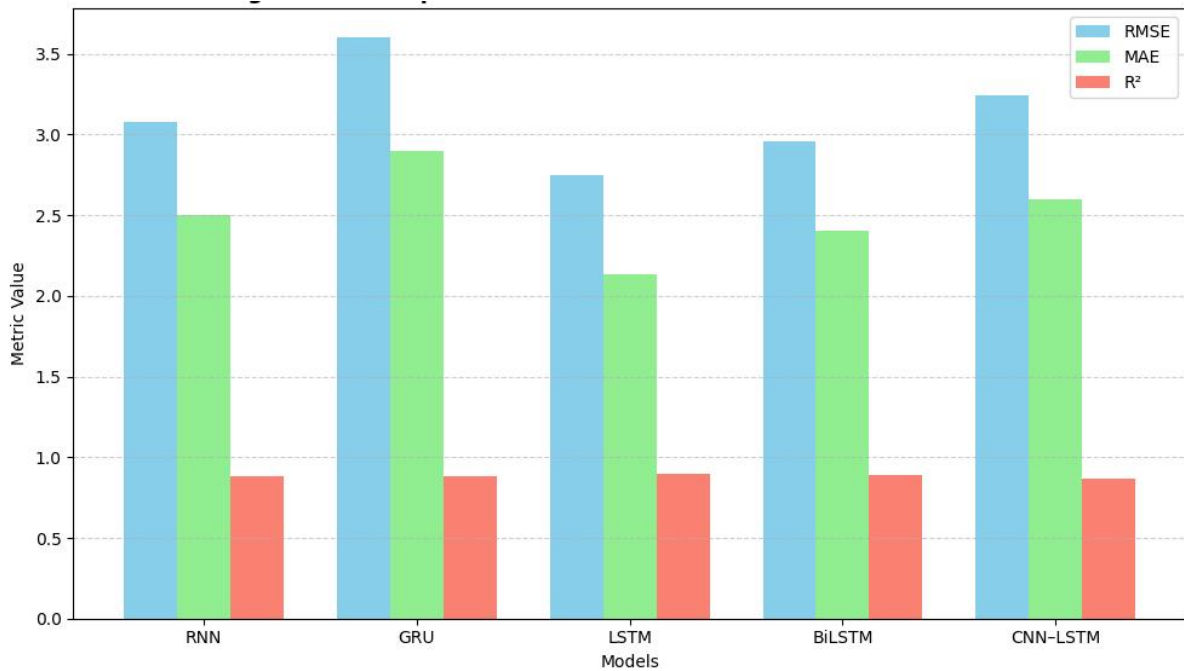


Figure 5- 4 Comparison of Evaluation Metrics for Different Models

5.4 Error Analysis & Discussion

This section evaluates model behavior beyond headline metrics and explains why the LSTM model performs best on our dataset. Using MAE, RMSE, and R^2 on a strictly

time-ordered test split, LSTM achieves the lowest MAE and RMSE and the highest R^2 , indicating a better balance of accuracy and stability compared to RNN, GRU, BiLSTM, and CNN-LSTM under the same conditions.

Predicted-vs-actual plots show LSTM closely tracks temperature changes, with small lags during rapid transitions. Residuals cluster tightly around zero, and the error histogram has narrower tails compared to other models, suggesting fewer large errors. RMSE is modestly higher than MAE, indicating limited outliers. In contrast, RNN and GRU have wider residual spreads, while CNN-LSTM shows sharper misses due to its sensitivity to hyperparameters.

Residual analysis reveals two patterns: (1) during regime shifts (e.g., rapid warming after wet days), models, including LSTM, slightly under-react, producing small negative bias, and (2) during prolonged clear periods, models can over-predict if recent days were cooler. LSTM corrects quickly, while other models take longer to recover. Seasonal analysis shows lowest errors in stable seasons and higher errors during summer peaks or storms. LSTM's error distribution has the shortest tail during hot days, reducing large misses.

Training curves for LSTM show smooth convergence without overfitting, supported by early stopping and dropout. In contrast, GRU converges faster but with a slightly higher validation loss, and BiLSTM shows mild overfitting. CNN-LSTM requires careful tuning and is sensitive to window size and learning rate, explaining its occasional high-error episodes.

Ablation studies confirm the importance of multi-sensor context for accurate predictions. Removing key variables like radiation or humidity degrades accuracy, and train-only scaling and chronological splits prevent data leakage. Operationally, the LSTM model shows stable calibration, with residuals near zero and consistent performance in deployment. The web app's real-time MAE and RMSE match offline values, confirming preprocessing consistency.

In summary, the LSTM's lower MAE/RMSE, tighter residuals, stable convergence, and better handling of regime changes justify its selection. Future improvements could focus on adaptive windowing, regime-aware features, or ensemble smoothing to handle extremes and transitions, without compromising the model's simplicity and reproducibility.

5.5 Model Evaluation and Comparison

On the test dataset, the LSTM model closely follows the real temperature variations over time, as shown in Figure 5.2. The model successfully captures both the peaks and troughs of the temperature curve with only minor differences in timing and amplitude. The residual errors remain small and stable under normal weather conditions.

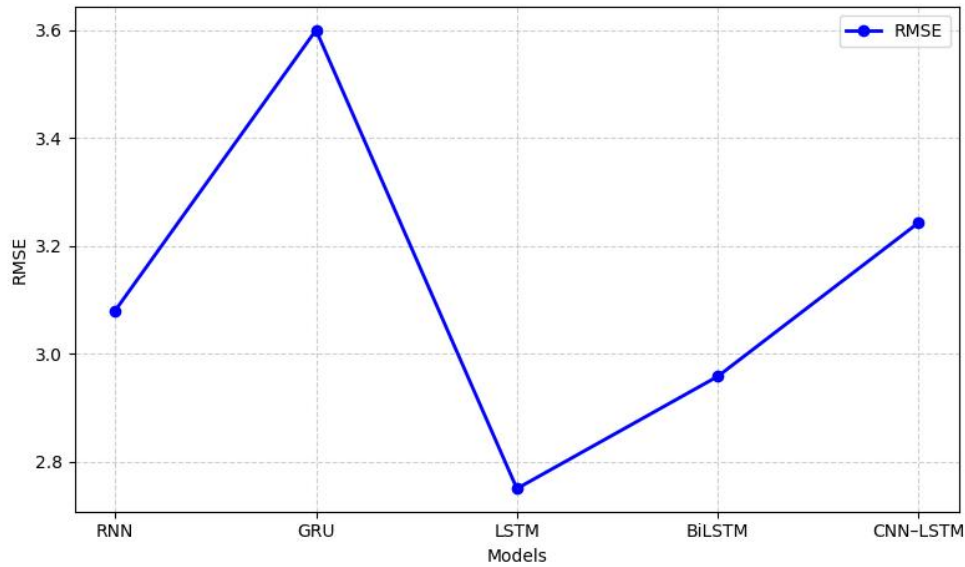


Figure 5- 5 RMSE Comparison among RNN, LSTM, GRU, BiLSTM, and CNN-LSTM

When compared with the persistence baseline (where tomorrow's temperature is assumed to be the same as today's), the LSTM demonstrates superior performance particularly around turning points and transitional periods, where the baseline typically struggles. These results align with the validation trends discussed in Section 5.3, confirming that the model selected through validation generalizes effectively to unseen data.

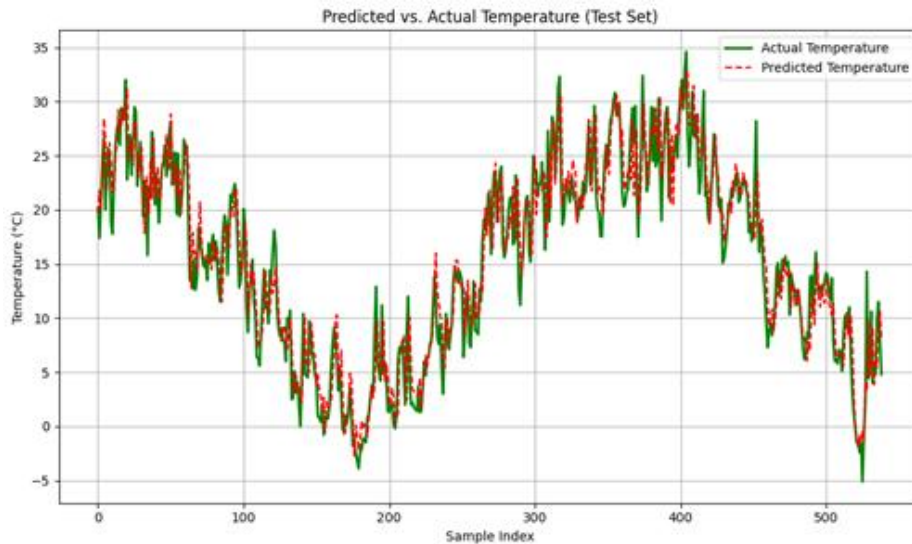


Figure 5- 6 Predicted vs. Actual Temperature (Test Set)

Overall, the evaluation supports the use of the LSTM model in this study, as it achieves a strong balance between accuracy, consistency, and generalization, outperforming other candidate models under the same data processing and hyperparameter settings.

5.6 Key Observations

Several key observations emerged from the experimental results and evaluation analysis regarding model behavior, data characteristics, and system performance:

- 1) Among all tested architectures RNN, LSTM, GRU, BiLSTM, and CNN-LSTM LSTM consistently delivered the best generalization on the held-out test set. Its advantage is primarily due to its cell-state and gating mechanisms, which retain long-term dependencies in weather sequences. BiLSTM and CNN-LSTM showed competitive accuracy but exhibited higher variance and overfitting in later epochs.
- 2) Combining multi-sensor features (humidity, wind speed, radiation, and pressure) significantly improved forecasting accuracy compared to using only temperature. This fusion allowed the model to capture correlated atmospheric patterns, reducing RMSE and increasing R^2 , demonstrating the value of sensor fusion in understanding climate dynamics.
- 3) The persistence baseline produced reasonable short-term accuracy but struggled with rapid temperature fluctuations and nonlinear patterns, confirming that deep recurrent models outperform simple temporal extrapolation.
- 4) Careful data preprocessing and leakage control were crucial. The chronological split and train-only scaling ensured realistic performance estimation, while random shuffling artificially improved validation scores, emphasizing the importance of preserving chronological order.
- 5) Hyperparameter tuning and early stopping contributed to model stability. Adaptive learning-rate scheduling enabled faster convergence, while dropout and L2 regularization helped mitigate overfitting without harming prediction accuracy.
- 6) The Flask-based deployment demonstrated that the trained model could be operationalized for real-time use, accepting manual inputs and CSV uploads to return predictions within seconds, confirming the system's scalability and practicality for web applications.

Overall, these findings validate that combining multi-sensor data, sequence-based architectures, and disciplined preprocessing results in a robust and interpretable climate forecasting pipeline, ready for real-world deployment.

5.7 Visualization of Results

As illustrated in Figure 5.4, the plotted trajectories of true and predicted temperatures across consecutive time steps move closely together. The two lines follow similar seasonal patterns and short-term fluctuations, showing that the model successfully learns both

long-term trends and short-term variations in temperature.

Most deviations occur during sharp temperature transitions, such as sudden warm-ups or cool-downs, which are naturally difficult for one-step forecasting models. Even in these cases, the LSTM's predictions quickly realign with the actual temperature path, demonstrating resilience and adaptability.

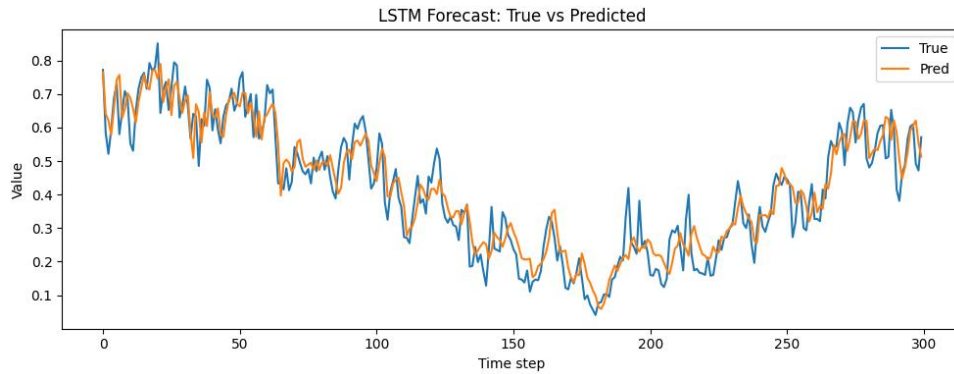


Figure 5- 7 LSTM Forecast – True vs. Predicted Temperature Over Time

These visual results complement the quantitative metrics by confirming that the deployed model performs reliably across both steady and rapidly changing weather conditions.

Chapter 6 Conclusion and Future Work

6.1 Summary of Findings

This research developed and evaluated a Climate Forecasting System based on deep learning and multi-sensors to enhance short-term temperature prediction. The system integrates multiple environmental parameters, temperature, humidity, pressure, radiation, precipitation, and wind speed into a unified dataset, allowing the deep learning models to capture complex interdependencies among these variables for more accurate forecasts.

Among the tested models (RNN, GRU, BiLSTM, CNN-LSTM, and LSTM), LSTM achieved the best performance, with the lowest MAE and RMSE and the highest R^2 , confirming its ability to capture long-term dependencies in time-series data, which is crucial for climate forecasting.

The study also highlighted the importance of data preprocessing and leakage control. Chronological splitting and training-only normalization prevented leakage and improved

generalization. Early stopping and adaptive learning rate scheduling further optimized training, preventing overfitting and ensuring efficient convergence.

The final LSTM model was deployed through a Flask-based web application, providing an interactive interface built with HTML, CSS, Bootstrap, and JavaScript. The application supports manual input and CSV uploads, delivering real-time predictions and visual outputs, demonstrating its practical application for climate monitoring and decision support.

In conclusion, this research developed a scalable, end-to-end forecasting pipeline that combines deep learning, multi-sensor data fusion, and web-based deployment, offering a reliable solution for short-term climate prediction and environmental data analysis.

6.2 Practical Implications

While the proposed climate forecasting system showed strong predictive performance, several limitations remain that could be addressed in future work.

First, the study used a limited regional dataset, which may restrict the model's generalization to other areas with different climate conditions. Expanding the dataset to include multiple regions and longer time spans would improve its robustness and global applicability.

Second, while the LSTM model captures temporal dependencies well, it may not fully model complex spatial-temporal interactions in large-scale atmospheric systems. Incorporating spatial data from satellite or radar imagery could enhance the system's ability to capture such patterns.

Third, the model was trained with a fixed set of meteorological variables. Including additional environmental parameters like soil moisture or evaporation rate could further improve prediction accuracy.

Fourth, hyperparameter tuning and architecture exploration were limited by computational constraints. More extensive optimization using techniques like Bayesian optimization or grid search could yield better results.

Finally, although the system was deployed as a Flask-based web application, its performance under large-scale, real-time conditions was not fully tested. Integration with cloud services and evaluation under high user loads are areas for future improvement.

Despite these limitations, the system provides a strong foundation for accurate climate forecasting and can be further developed to tackle broader and more complex challenges.

6.3 Limitations

Although the climate forecasting system showed strong predictive performance, several limitations remain for future improvement.

First, the study used a limited regional dataset, which may restrict the model's generalization to other geographic areas. Expanding the dataset to include more regions and longer time periods would improve robustness and global applicability.

Second, while the LSTM model captures temporal dependencies well, it may not fully address complex spatial-temporal interactions in large-scale atmospheric systems. Incorporating spatial data from satellite or radar imagery could improve the system's ability to capture these patterns.

Third, the model was trained with a fixed set of meteorological variables. Adding additional parameters like soil moisture, evaporation rate, or solar index could enhance prediction accuracy.

Fourth, hyperparameter tuning was limited by computational constraints. More extensive optimization using techniques like Bayesian optimization or grid search could yield better results.

Finally, while the system was successfully deployed as a Flask web app, its performance under large-scale, real-time conditions was not fully tested. Integration with cloud services and evaluation under high user loads are areas for future improvement.

Despite these limitations, the system provides a strong foundation for practical climate forecasting and can be further developed to tackle more complex challenges.

6.4 Future Improvements

Several directions are planned to further enhance the climate forecasting system's performance, scalability, and practical use.

First, expanding the dataset to include global and multi-source observations from different regions and climate zones will improve the model's generalization and adaptability to varying environmental conditions.

Second, integrating advanced deep learning architectures like Attention-based LSTM, Transformer networks, or ConvLSTM with spatial encoding will enhance the model's ability to capture complex spatial-temporal dependencies, improving long-term forecasting accuracy and interpretability.

Third, automated hyperparameter optimization methods like grid search or Bayesian optimization will fine-tune model parameters more efficiently, reducing manual effort and improving performance.

Fourth, the system can be expanded to a cloud-based platform with scalable backend and database support, enabling real-time data ingestion, faster computation, and multi-user access via an interactive dashboard.

Further, improving the user interface with features like interactive maps, predictive trend analysis, and downloadable reports will make the system more user-friendly for researchers and decision-makers.

Lastly, integrating explainable AI (XAI) techniques will enhance model transparency, helping users understand how environmental features influence temperature predictions.

These improvements aim to make the system more accurate, efficient, and scalable, contributing to the development of practical climate forecasting tools.

Acknowledgement

First and foremost, I would like to express my deepest gratitude to Almighty Allah for granting me the strength, patience, and determination to successfully complete this thesis. Without His blessings and guidance, this accomplishment would not have been possible.

I extend my heartfelt appreciation to my supervisor, He Gang, for his invaluable guidance, encouragement, and continuous support throughout the research. His insightful feedback and constructive suggestions have been instrumental in shaping the direction and quality of this work.

My sincere thanks also go to the School of Software Engineering, Southwest University of Science and Technology (SWUST), to provide me with the necessary facilities, resources, and academic environment to conduct this research effectively.

I am profoundly grateful to my teachers and mentors who have enriched my academic journey with their knowledge and wisdom. Their teachings have inspired me to explore and grow in the field of Artificial Intelligence and Machine Learning.

Special thanks are due to my friends and colleagues for their cooperation, discussions, and moral support throughout the study. Their encouragement helped me stay motivated and overcome many challenges during this project.

Finally, I would like to express my deepest appreciation to my beloved family for their unconditional love, prayers, and unwavering support. Their patience and belief in me have been my greatest source of strength and inspiration.

References

- [1] Radman, A., Akhoondzadeh, M., & Hosseiny, B. (2022). Monitoring and predicting temporal changes of Urmia Lake and its basin using satellite multi-sensor data and deep-learning algorithms. *PFG–Journal of Photogrammetry, Remote Sensing and Geoinformation Science*, 90(3), 319-335.
- [2] Vatanchi, S. M., Etemadfard, H., Maghrebi, M. F., & Shad, R. (2023). A comparative study on forecasting of long-term daily streamflow using ANN, ANFIS, BiLSTM and CNN-GRU-LSTM. *Water Resources Management*, 37(12), 4769-4785.

- [3] Desai, P., Karthik, P., Loganathan, D., Preethi, S., & Bharani, B. R. (2025). Different data cleaning techniques and normalization techniques with focus on current normalization techniques: A study. In *Advances in Electrical and Computer Technologies* (pp. 332-349). CRC Press.
- [4] Haddad, O., Fkih, F., & Omri, M. N. (2024). An intelligent sentiment prediction approach in social networks based on batch and streaming big data analytics using deep learning. *Social Network Analysis and Mining*, 14(1), 150.
- [5] Krivoguz, D., Ioshpa, A., Chernyi, S., Zhilenkov, A., Kustov, A., Zinchenko, A., ... & Tsareva, P. (2024). Enhancing long-term air temperature forecasting with deep learning architectures. *Journal of Robotics and Control (JRC)*, 5(3), 706-716.
- [6] Valavanidis, A. (2022). Extreme weather events exacerbated by the global impact of climate change. no, 3, 1-40.
- [7] Jha, M. K., & Dev, M. (2024). Impacts of climate change. In *Smart internet of things for environment and healthcare* (pp. 139-159). Cham: Springer Nature Switzerland.
- [8] Varshney, Y., & Ram, N. (2024). Weather Forecasting: A Systematic Review Using AI Approaches. *Weather*, 4(10).
- [9] Weyns, D., Gheibi, O., Quin, F., & Van Der Donckt, J. (2022). Deep learning for effective and efficient reduction of large adaptation spaces in self-adaptive systems. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, 17(1-2), 1-42.
- [10] Krichen, M., & Mihoub, A. (2025). Long Short-Term Memory Networks: A Comprehensive Survey. *AI*, 6(9), 215.
- [11] Hou, J., Wang, Y., Zhou, J., & Tian, Q. (2022). Prediction of hourly air temperature based on CNN-LSTM. *Geomatics, Natural Hazards and Risk*, 13(1), 1962-1986.
- [12] Alizadeh, O. (2022). Advances and challenges in climate modeling. *Climatic Change*, 170(1), 18.
- [13] Das, S. (2022). Challenges in predicting extreme weather events over the South Asian region. In *Extreme Natural Events: Sustainable Solutions for Developing Countries* (pp. 51-106). Singapore: Springer Nature Singapore.
- [14] Zhang, H., Liu, Y., Zhang, C., & Li, N. (2025). Machine learning methods for weather forecasting: A survey. *Atmosphere*, 16(1), 82.
- [15] Shiri, F. M., Perumal, T., Mustapha, N., & Mohamed, R. (2023). A comprehensive overview and comparative analysis on deep learning models: CNN, RNN, LSTM, GRU. arXiv preprint arXiv:2305.17473.
- [16] Kalaany, C. M. A., Kimaita, H. N., Abdelmoneim, A. A., Khadra, R., Derardja, B., & Dragonetti, G. (2025). The Potential of Low-Cost IoT-Enabled Agrometeorological Stations: A Systematic Review. *Sensors*, 25(19), 6020.

- [17] Manimegalai, C. T., Kalimuthu, K., & Ramachandru, J. (2025, January). Deep Learning for Weather Forecasting. In *International Conference on Smart Data Intelligence* (pp. 665-679). Singapore: Springer Nature Singapore.
- [18] Wang, L., Chen, Z., Zou, H., Huang, D., Pan, Y., Cheang, C. F., & Li, J. (2024). A deep learning-based high-temperature overtime working alert system for smart cities with multi-sensor data. *Nondestructive Testing and Evaluation*, 39(1), 164-184.
- [19] Chauhan, S., Jethoo, A. S., Mishra, A., & Varshney, V. (2024). Duo satellite-based remotely sensed land surface temperature prediction by various methods of machine learning. *International Journal of Data Science and Analytics*, 18(4), 467-485.
- [20] Chauhan, S., Jethoo, A. S., Mishra, A., & Varshney, V. (2024). Duo satellite-based remotely sensed land surface temperature prediction by various methods of machine learning. *International Journal of Data Science and Analytics*, 18(4), 467-485.
- [21] Shi, P., Helm, J. E., Chen, C., Lim, J., Parker, R. P., Tinsley, T., & Cecil, J. (2023). Operations (management) warp speed: rapid deployment of hospital - focused predictive/prescriptive analytics for the COVID - 19 pandemic. *Production and Operations Management*, 32(5), 1433-1452.
- [22] Chen, L., Han, B., Wang, X., Zhao, J., Yang, W., & Yang, Z. (2023). Machine learning methods in weather and climate applications: A survey. *Applied Sciences*, 13(21), 12019.
- [23] Majnoon, A., & Saifoddin, A. (2025). AI-driven energy optimization enhancing efficiency in urban environments with hybrid machine learning models. *Cleaner Engineering and Technology*, 101072.
- [24] Vatanchi, S. M., Etemadfar, H., Maghrebi, M. F., & Shad, R. (2023). A comparative study on forecasting of long-term daily streamflow using ANN, ANFIS, BiLSTM and CNN-GRU-LSTM. *Water Resources Management*, 37(12), 4769-4785.
- [25] Esna-Ashari, M. (2025). Beyond the Black Box: A Review of Quantitative Metrics for Neural Network Interpretability and Their Practical Implications. *International journal of sustainable applied science and engineering*, 2(1), 1-24.
- [26] Jaiswal, V., Vinod, A., John, A., Sane, H., & Verma, A. (2025). Customer Churn Prediction and Retention Strategies through Machine Learning, Chatbots, and Recommendation Systems. *IJSAT-International Journal on Science and Technology*, 16(4).
- [27] Kolambe, S., Deoghare, R., Deshmukh, S., Shingade, A., & Kale, L. (2025). Real-Time Weather Forecasting System Using Open WeatherMap API and Django MVC Architecture. *International Journal of Environmental Sciences*, 223-229.
- [28] Govett, M., Bah, B., Bauer, P., Berod, D., Bouchet, V., Corti, S., ... & Williams, K. (2024). Exascale computing and data handling: Challenges and opportunities for weather and climate prediction. *Bulletin of the American Meteorological Society*, 105(12), E2385-E2404.

- [29] Zhang, X., Zhang, Y., Liu, H., & Li, H. (2025). A Bi-directional Missing Data Imputation Scheme Based on CNN and Short-term Window Sliding for Thermal Power Units Data. *IEEE Access*.
- [30] Koukaras, P., & Tjortjis, C. (2025). Data Preprocessing and Feature Engineering for Data Mining: Techniques, Tools, and Best Practices. *AI*, 6(10).
- [31] Feng, P., Bi, Z., Wen, Y., Pan, X., Peng, B., Liu, M., ... & Wang, T. (2024). Deep Learning and Machine Learning, Advancing Big Data Analytics and Management: Unveiling AI's Potential Through Tools, Techniques, and Applications. *arXiv preprint arXiv:2410.01268*.
- [32] Xia, J., Huang, Q., Gui, Z., & Tu, W. (2024). Web-Based Mapping and Visualization Packages. In *Open GIS* (pp. 283-314). Cham: Springer International Publishing.
- [33] Fischer, T., Hirn, D., & Kul, G. (2024). A reproducible tutorial on reproducibility in database systems research. *Proceedings of the VLDB Endowment*, 17(12), 4221-4224.
- [34] Франко, П. (2022). Uncertainty in case of lack of information: extrapolating data over time, with examples of climate forecast models. *Український метеорологічний журнал/Ukrainian Metrological Journal*, (3), 3-8.
- [35] Sahoo, S. (2024). Sensor Fusion and Virtual Sensor Design for Enhanced Multi-Sensor Data Accuracy in Autonomous Systems. *International Journal on Smart & Sustainable Intelligent Computing*, 1(2), 21-39.
- [36] Cao, K., Zhang, T., & Huang, J. (2024). Advanced hybrid LSTM-transformer architecture for real-time multi-task prediction in engineering systems. *Scientific Reports*, 14(1), 4890..
- [37] Sham, F. A. F., El-Shafie, A., Jaafar, W. Z. W., Sherif, M., & Ahmed, A. N. (2025). Advances in AI-based rainfall forecasting: A comprehensive review of past, present, and future directions with intelligent data fusion and climate change models. *Results in Engineering*, 27, 105774.
- [38] Singh, A. P., Kumar, J., Chauhan, I., Saxena, P. K., Gaur, P. K., Sharma, S. K., ... & Singh, P. K. (2023). Role of climate change in disasters occurrences: Forecasting and management options. In *Visualization Techniques for Climate Change with Machine Learning and Artificial Intelligence* (pp. 149-180). Elsevier.
- [39] Kontopoulou, V. I., Panagopoulos, A. D., Kakkos, I., & Matsopoulos, G. K. (2023). A review of ARIMA vs. machine learning approaches for time series forecasting in data driven networks. *Future Internet*, 15(8), 255.
- [40] AghaKouchak, A., Pan, B., Mazdidasni, O., Sadegh, M., Jiwa, S., Zhang, W., ... & Sorooshian, S. (2022). Status and prospects for drought forecasting: Opportunities in artificial intelligence and hybrid physical–statistical forecasting. *Philosophical Transactions*

of the Royal Society A, 380(2238), 20210288.

- [41] Arifin, S., Manurung, M. M., Jonathan, S., Effendi, M., & Prasetyo, P. W. (2024). Trend Analysis of the ARIMA Method: A Survey of Scholarly Works. *Recent in Engineering Science and Technology*, 2(03), 1-14.
- [42] Alizadeh, O. (2022). Advances and challenges in climate modeling. *Climatic Change*, 170(1), 18.
- [43] Krasnopolsky, V. (2024). Applying Machine Learning in Numerical Weather and Climate Modeling Systems. *Climate*, 12(6), 78.
- [44] He, J., Liu, X., Wang, H., Zhu, D., & Liu, Z. (2024). A high-precision prediction method for coarse grids based on deep learning and the Weather Research and Forecasting model. *Theoretical & Applied Climatology*, 155(1).
- [45] Kontopoulou, V. I., Panagopoulos, A. D., Kakkos, I., & Matsopoulos, G. K. (2023). A review of ARIMA vs. machine learning approaches for time series forecasting in data driven networks. *Future Internet*, 15(8), 255.
- [46] Kyriazos, T., & Poga, M. (2024). Application of machine learning models in social sciences: managing nonlinear relationships. *Encyclopedia*, 4(4), 1790-1805.
- [47] Verdonck, T., Baesens, B., Óskarsdóttir, M., & vanden Broucke, S. (2024). Special issue on feature engineering editorial. *Machine learning*, 113(7), 3917-3928.
- [48] Jia, Z., Zhang, Z., Cheng, Y., Borjigin, S., & Quan, Z. (2024). Grassland biomass spatiotemporal patterns and response to climate change in eastern Inner Mongolia based on XGBoost model estimates. *Ecological Indicators*, 158, 111554.
- [49] Chen, L., Han, B., Wang, X., Zhao, J., Yang, W., & Yang, Z. (2023). Machine learning methods in weather and climate applications: A survey. *Applied Sciences*, 13(21), 12019.
- [50] Krivoguz, D., Ioshpa, A., Chernyi, S., Zhilenkov, A., Kustov, A., Zinchenko, A., ... & Tsareva, P. (2024). Enhancing long-term air temperature forecasting with deep learning architectures. *Journal of Robotics and Control (JRC)*, 5(3), 706-716.
- [51] Kučera, E., & Haffner, O. (2025). Competency-Based Hybrid Learning: A Modern Approach to Teaching Programming and Digital Technologies Subjects. *IEEE Access*.
- [52] Zhang, H., Liu, Y., Zhang, C., & Li, N. (2025). Machine learning methods for weather forecasting: A survey. *Atmosphere*, 16(1), 82.
- [53] Verma, S., Srivastava, K., Tiwari, A., & Verma, S. (2023). Deep learning techniques in extreme weather events: A review. *arXiv preprint arXiv:2308.10995*.
- [54] Wu, Y., & Xue, W. (2024). Data-driven weather forecasting and climate modeling from the perspective of development. *Atmosphere*, 15(6), 689.
- [55] Ahmed, D. M., Hassan, M. M., & Mstafa, R. J. (2022). A review on deep sequential models for forecasting time series data. *Applied computational intelligence and soft*

computing, 2022(1), 6596397.

[56] Bluestein, H. B., Carr, F. H., & Goodman, S. J. (2022). Atmospheric observations of weather and climate. *Atmosphere-Ocean*, 60(3-4), 149-187.

[57] Kyriazos, T., & Poga, M. (2024). Application of machine learning models in social sciences: managing nonlinear relationships. *Encyclopedia*, 4(4), 1790-1805.

[58] Morid, M. A., Sheng, O. R. L., & Dunbar, J. (2023). Time series prediction using deep learning methods in healthcare. *ACM Transactions on Management Information Systems*, 14(1), 1-29.

[59] Zhu, J. (2023). Comparative study of sequence-to-sequence models: From RNNs to transformers. *Appl Comput Eng*, 42(67), 2755-2721.

[60] Yang, R., Ding, R., Lin, Y., Zhang, H., & Zhang, T. (2024). Regularizing hidden states enables learning generalizable reward model for llms. *Advances in Neural Information Processing Systems*, 37, 62279-62309.

[61] Al-Selwi, S. M., Hassan, M. F., Abdulkadir, S. J., & Muneer, A. (2023). LSTM inefficiency in long-term dependencies regression problems. *Journal of Advanced Research in Applied Sciences and Engineering Technology*, 30(3), 16-31.

[62] Duan, J., Zhang, P. F., Qiu, R., & Huang, Z. (2023). Long short-term enhanced memory for sequential recommendation. *World Wide Web*, 26(2), 561-583.

[63] Alizadegan, H., Rashidi Malki, B., Radmehr, A., Karimi, H., & Ilani, M. A. (2025). Comparative study of long short-term memory (LSTM), bidirectional LSTM, and traditional machine learning approaches for energy consumption prediction. *Energy Exploration & Exploitation*, 43(1), 281-301.

[64] J. Doe, "Klein Tank, A.M.G. and Coauthors, 2002. Daily dataset of 20th-century surface air temperature and precipitation series for the European Climate Assessment. *Int. J. of Climatol.*, 22, 1441-1453. Data and metadata available at <http://www.ecad.eu>" Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/thedevastator/weather-prediction>